

BAB III

PELAKSANAAN KERJA PROFESI

3.1 Bidang Kerja

Selama melaksanakan Kerja Profesi di PT Reliable Future Technology (RFT), praktikan ditempatkan pada bidang Frontend Development untuk proyek pembuatan website e-commerce sayur-sayuran bernama PureGreens. Fokus pekerjaan berada pada proses implementasi tampilan menggunakan Laravel, tanpa terlibat dalam pembuatan desain UI/UX.

Tugas utama yang dilakukan meliputi pembuatan struktur halaman, penerapan styling menggunakan TailwindCSS, penyesuaian layout agar responsif, serta menampilkan data yang diteruskan dari backend. Praktikan juga memastikan setiap halaman dapat berjalan dengan baik dan tampil konsisten sesuai kebutuhan sistem.

3.2 Pelaksanaan Kerja

Pelaksanaan Kerja Profesi dilakukan dengan mengikuti alur kerja yang berlaku di PT Reliable Future Technology (RFT). Selama kegiatan, praktikan terlibat dalam proses pengembangan bagian frontend untuk proyek website e-commerce PureGreens, sebuah platform penjualan sayur-sayuran berbasis Laravel. Aktivitas kerja dilakukan secara bertahap, dimulai dari memahami kebutuhan sistem, menyusun rancangan dasar, hingga mengimplementasikan tampilan halaman sesuai fungsi yang dibutuhkan. Setiap tugas yang dikerjakan tetap berkoordinasi dengan tim pengembang lain agar hasil frontend dapat terhubung dengan backend dan berjalan sebagaimana mestinya.

3.2.1 Analisis Kebutuhan Sistem

Analisis kebutuhan sistem dilakukan untuk mengidentifikasi fitur utama yang harus didukung oleh tampilan website e-commerce (Mahardika et al., 2025). Analisis ini menjadi dasar dalam penyusunan struktur halaman dan implementasi frontend menggunakan Laravel. Kebutuhan sistem dibagi menjadi dua kategori utama, yaitu kebutuhan fungsional dan non-fungsional.

a. Kebutuhan Fungsional

Kebutuhan fungsional menggambarkan fitur dasar yang harus tersedia pada aplikasi untuk memenuhi kebutuhan pengguna (Iskandar & Ratnasari, 2023). Berdasarkan arahan dan tampilan yang dikembangkan, kebutuhan tersebut meliputi:

- Halaman login dan registrasi untuk autentikasi pengguna.
- Halaman beranda yang menampilkan daftar kategori serta produk beserta harga dan gambar.
- Halaman produk rekomendasi yang menampilkan rekomendasi secara visual.
- Fitur keranjang belanja untuk menambahkan, mengubah jumlah, dan menghapus produk.
- Halaman riwayat pesanan untuk melihat transaksi pengguna.
- Halaman profil pengguna untuk memperbarui informasi akun dan password.

b. Kebutuhan Non-Fungsional

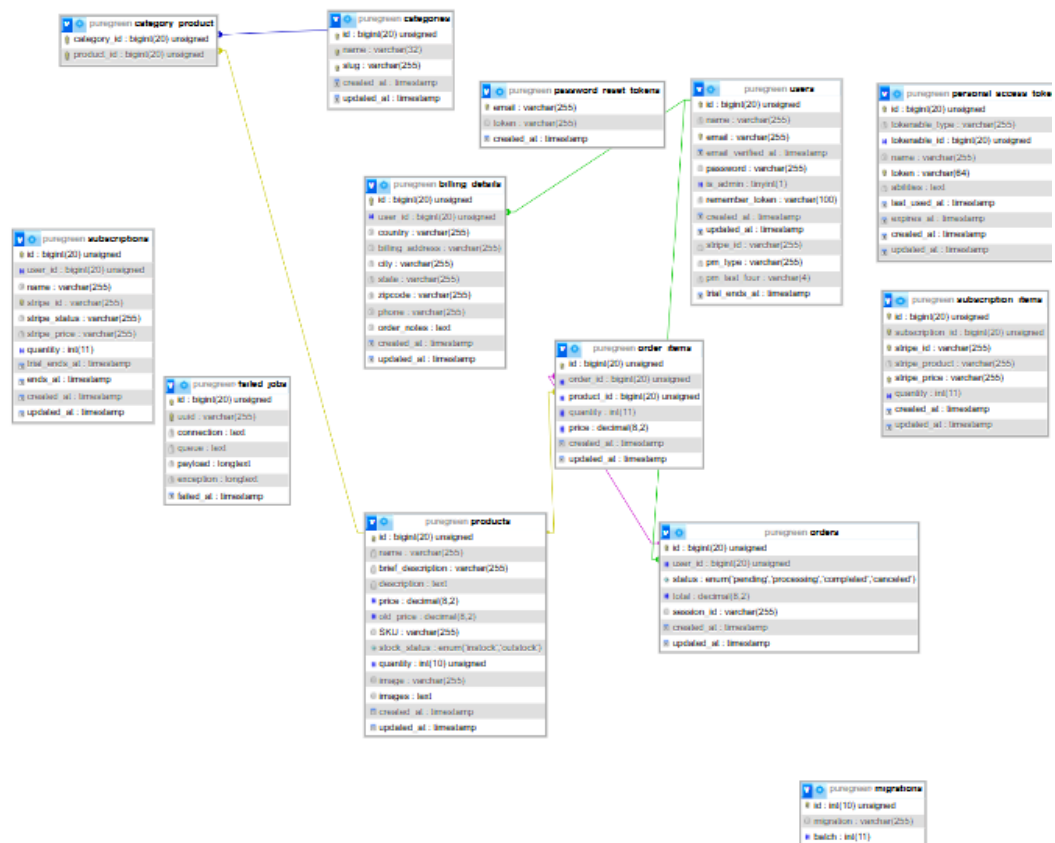
Kebutuhan non-fungsional berfokus pada kualitas tampilan, pengalaman penggunaan, serta performa website (Qadri, 2023). Beberapa kebutuhan tersebut antara lain:

- Antarmuka yang rapi, sederhana, dan mudah dipahami oleh pengguna.
- Konsistensi layout, warna, ukuran elemen, dan gaya visual pada seluruh halaman.
- Navigasi yang jelas dan mudah diakses, terutama pada bagian menu akun dan keranjang.
- Tampilan yang responsif pada berbagai ukuran layar.
- Waktu muat halaman yang cepat agar interaksi pengguna lebih nyaman.

3.2.2 Perancangan

Perancangan dilakukan untuk memetakan alur interaksi pengguna serta struktur proses yang akan diimplementasikan pada tampilan website PureGreens (Aditya, 2022). Tahap ini bertujuan agar setiap fitur memiliki alur kerja yang jelas sebelum dikembangkan pada bagian frontend (Ekasmara, 2020). Perancangan mencakup pembuatan Mockup Use Case Diagram untuk menggambarkan

hubungan antara aktor dan fungsionalitas sistem, serta Sequence Diagram untuk menunjukkan urutan proses yang terjadi saat pengguna berinteraksi dengan aplikasi.

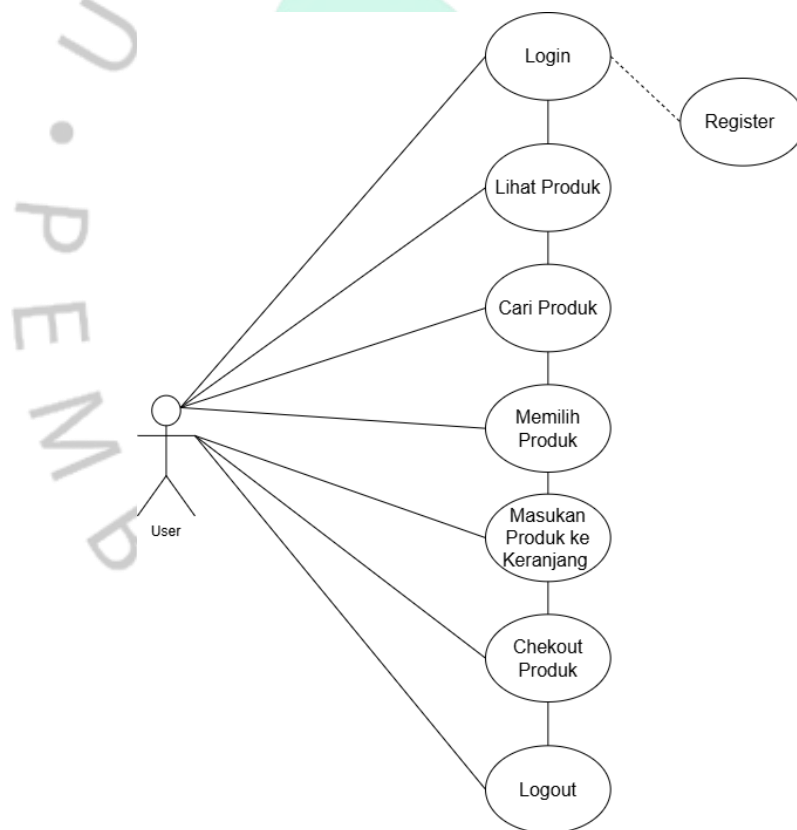


Gambar 3.31 Entity Relationship Diagram (ERD)

Gambar di atas menunjukkan struktur basis data aplikasi e-commerce PureGreens yang terdiri dari beberapa tabel utama yang saling terhubung dalam mendukung proses penjualan produk secara daring. Tabel users menyimpan data akun pengguna seperti id, name, email, password, dan informasi autentikasi lainnya, yang kemudian berelasi dengan tabel orders untuk mencatat pesanan pengguna, termasuk total_amount dan status transaksi. Setiap pesanan memiliki detail item yang disimpan dalam tabel order_items berisi product_id, quantity, dan price. Produk yang dijual tersimpan pada tabel products dengan atribut seperti nama, deskripsi, harga, stok, dan gambar, yang dikelompokkan dalam tabel categories, dengan hubungan many-to-many melalui tabel penghubung

category_product. Selain itu, informasi pengiriman tercatat dalam tabel billing_details yang berelasi one-to-one dengan orders melalui order_id. Struktur database ini juga didukung oleh tabel bawaan Laravel seperti password_reset_tokens untuk pemulihan akses, personal_access_tokens untuk otentikasi API, serta migrations dan failed_jobs untuk pemeliharaan sistem. Seluruh relasi yang diterapkan memperhatikan prinsip normalisasi dan integritas data dengan pemanfaatan foreign key agar sistem dapat beroperasi secara optimal, konsisten, dan mendukung proses bisnis aplikasi PureGreens sesuai kebutuhan fungsionalnya.

Gambar 3.1 Use Case Diagram



Use Case Diagram menggambarkan hubungan antara pengguna dan fungsionalitas utama yang tersedia pada website PureGreens. Pada diagram tersebut, terdapat satu aktor yaitu User yang dapat mengakses berbagai fitur, mulai dari proses autentikasi (Login dan Register), melihat beranda, menelusuri produk, mencari produk tertentu, hingga mengelola keranjang belanja. Selain itu,

user juga dapat melakukan proses checkout, memperbarui informasi profil, dan melihat riwayat pesanan yang pernah dilakukan. Diagram ini menjadi dasar untuk memastikan setiap fungsi pada sistem memiliki alur interaksi yang jelas sebelum masuk ke tahap implementasi.

Tabel 3.8 Deskripsi Use Case Login

Elemen	Deskripsi
Nama Use-case	Login
Aktor	User
Deskripsi	User melakukan proses autentikasi agar dapat mengakses fitur utama aplikasi.
Normal Course	1.User memasukkan email dan password. 2.Sistem memvalidasi informasi login. 3.Jika benar, sistem mengarahkan ke halaman utama.
Alternate Course	Jika data tidak valid, sistem menampilkan pesan kesalahan.
Pre-Condition	User sudah memiliki akun.
Post- condition	User berhasil masuk ke sistem.
Assumption	User memahami akun login yang digunakan.

Tabel 3.9 Deskripsi Use Case Register

Elemen	Deskripsi
Nama Use-case	Register
Aktor	User

Deskripsi	User membuat akun baru agar dapat menggunakan layanan pemesanan.
Normal Course	1.User mengisi formulir pendaftaran. 2.Sistem menyimpan data dan membuat akun baru. 3. User dapat melakukan login..
Alternate Course	Email sudah terdaftar, sistem menampilkan peringatan..
Pre-Condition	User belum memiliki akun.
Post- condition	Akun baru berhasil dibuat.
Assumption	Data yang diinput valid.

Tabel 3.10 Deskripsi Use Case Lihat Produk

Elemen	Deskripsi
Nama Use-case	Lihat Produk
Aktor	User
Deskripsi	User melihat daftar produk yang tersedia pada sistem.
Normal Course	1.User mengakses halaman produk. 2. Sistem menampilkan daftar produk secara dinamis.
Alternate Course	-
Pre-Condition	User telah login.
Post- condition	Produk tampil di layar.
Assumption	Data yang diinput valid.

Tabel 3.10 Deskripsi Use Case Lihat Produk

Tabel 3.11 Deskripsi Use Case Cari Produk

Elemen	Deskripsi
Nama Use-case	Cari Produk
Aktor	User
Deskripsi	User dapat melakukan pencarian produk berdasarkan nama atau kategori.
Normal Course	1.User memasukkan kata kunci pencarian. 2. Sistem menampilkan produk yang sesuai.
Alternate Course	Produk tidak ditemukan, sistem menampilkan pesan “Produk tidak tersedia”.
Pre-Condition	Produk tersedia di database.
Post- condition	Daftar produk hasil pencarian tampil.
Assumption	Kata kunci sesuai dengan data produk.

Tabel 3.12 Deskripsi Use Case Memilih Produk

Elemen	Deskripsi
Nama Use-case	Memilih Produk
Aktor	User
Deskripsi	User memilih salah satu produk untuk melihat detailnya .
Normal Course	1.User memilih produk dari daftar. 2. Sistem menampilkan detail produk.

Alternate Course	Email sudah terdaftar, sistem menampilkan peringatan..
Pre-Condition	Produk tersedia.
Post- condition	Detail produk tampil.
Assumption	Sistem dapat mengambil informasi produk.

Tabel 3.13 Deskripsi Use Case Masukan Produk ke Keranjang

Elemen	Deskripsi
Nama Use-case	Masukkan Produk ke Keranjang
Aktor	User
Deskripsi	User menambahkan produk ke keranjang sebelum melakukan checkout.
Normal Course	1. User menekan tombol tambah ke keranjang. 2. Sistem memperbarui data keranjang belanja.
Alternate Course	Stok tidak mencukupi → Sistem menampilkan pesan "Stok habis".
Pre-Condition	User sedang melihat produk.
Post- condition	Produk tersimpan di keranjang belanja.
Assumption	Stok tersedia.

Tabel 3.14 Deskripsi Use Case Chekout Produk

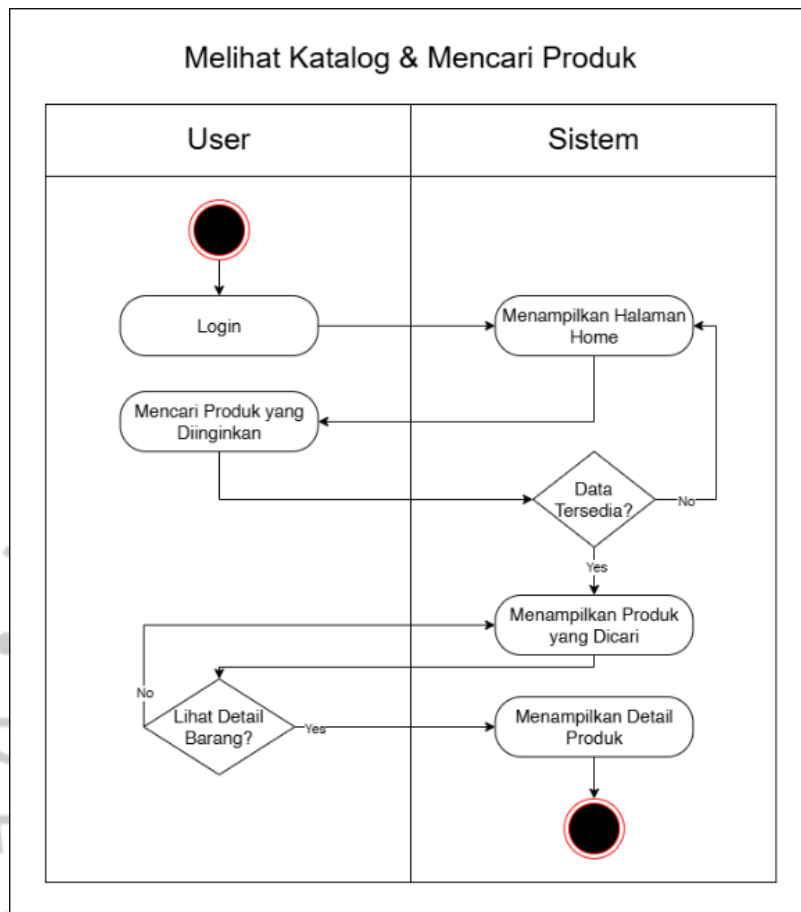
Elemen	Deskripsi
Nama Use-case	Checkout Produk
Aktor	User
Deskripsi	User menyelesaikan transaksi atas produk yang sudah ada di keranjang.
Normal Course	1. User membuka halaman checkout. 2. User memasukkan informasi pengiriman. 3. Sistem menyimpan pesanan sebagai transaksi.
Alternate Course	Pembayaran gagal → Sistem membatalkan transaksi.
Pre-Condition	Produk sudah ditambahkan ke keranjang.
Post- condition	Pesanan tersimpan di database.
Assumption	Metode pembayaran valid.

Tabel 3.15 Deskripsi Use Case Lohout

Elemen	Deskripsi
Nama Use-case	Logout
Aktor	User
Deskripsi	User keluar dari akun untuk mengakhiri sesi.

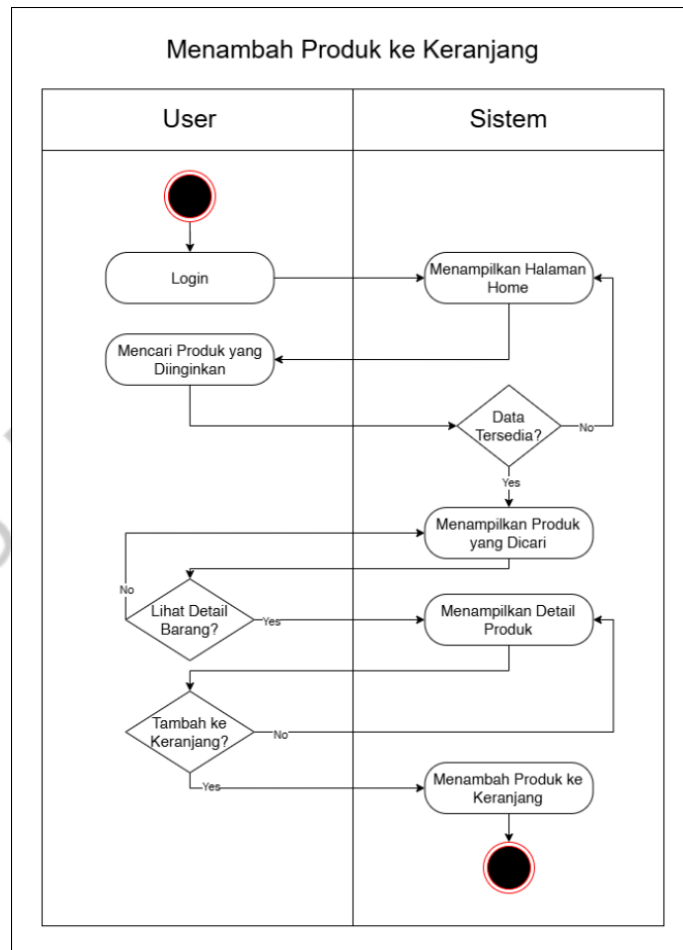
Normal Course	1. User menekan tombol logout. 2. Sistem menghapus session login dan kembali ke halaman awal.
Alternate Course	-
Pre-Condition	User telah login.
Post- condition	Sesi pengguna berakhir.
Assumption	Session user masih aktif sebelumnya.

● Activity diagram digunakan untuk memodelkan alur aktivitas yang terjadi pada sistem PureGreens dari perspektif pengguna dan sistem. Setiap diagram menggambarkan rangkaian langkah yang dilakukan pengguna, keputusan yang memengaruhi alur proses, serta respons yang diberikan sistem pada setiap tahap. Dengan adanya activity diagram, perancangan alur kerja menjadi lebih terstruktur, mudah dianalisis, serta membantu memastikan bahwa setiap proses berjalan secara logis sebelum diimplementasikan pada bagian frontend.



Gambar 3.10 Activity Diagram Melihat Katalog & Mencari Produk

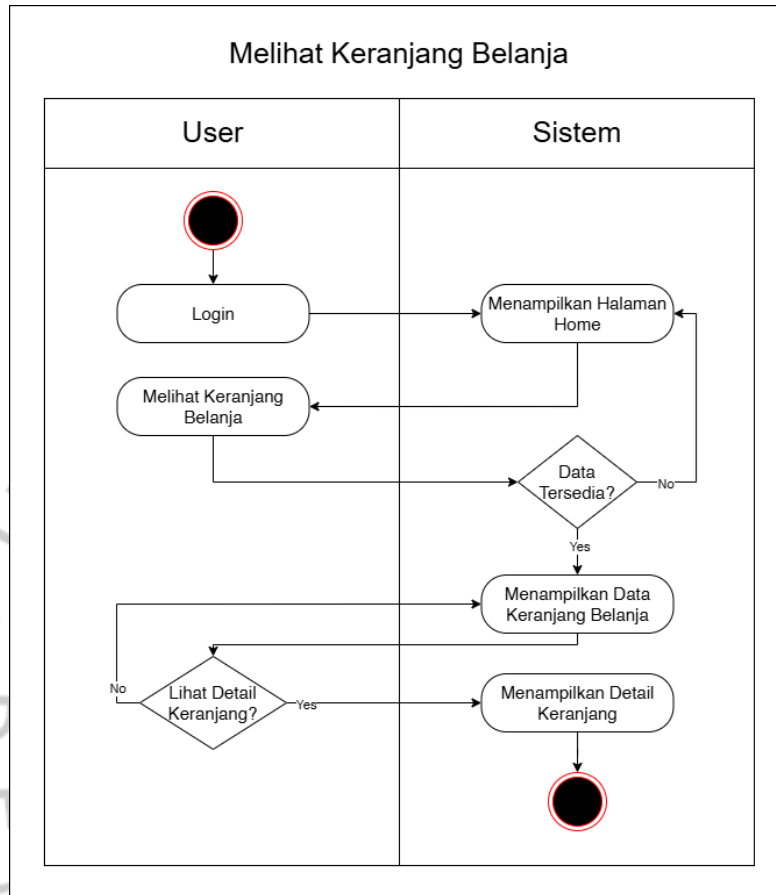
Activity diagram ini menggambarkan proses ketika pengguna ingin melihat daftar produk atau mencari produk tertentu pada halaman katalog. Setelah pengguna melakukan login, sistem menampilkan halaman utama dan memeriksa ketersediaan data produk. Jika data tersedia, sistem menampilkan daftar produk sesuai permintaan pengguna. Apabila pengguna ingin melihat detail produk, sistem akan menampilkan informasi lengkap produk yang dipilih sebelum proses berakhir.



Gambar 3.11 Activity Diagram Menambahkan Produk ke Keranjang

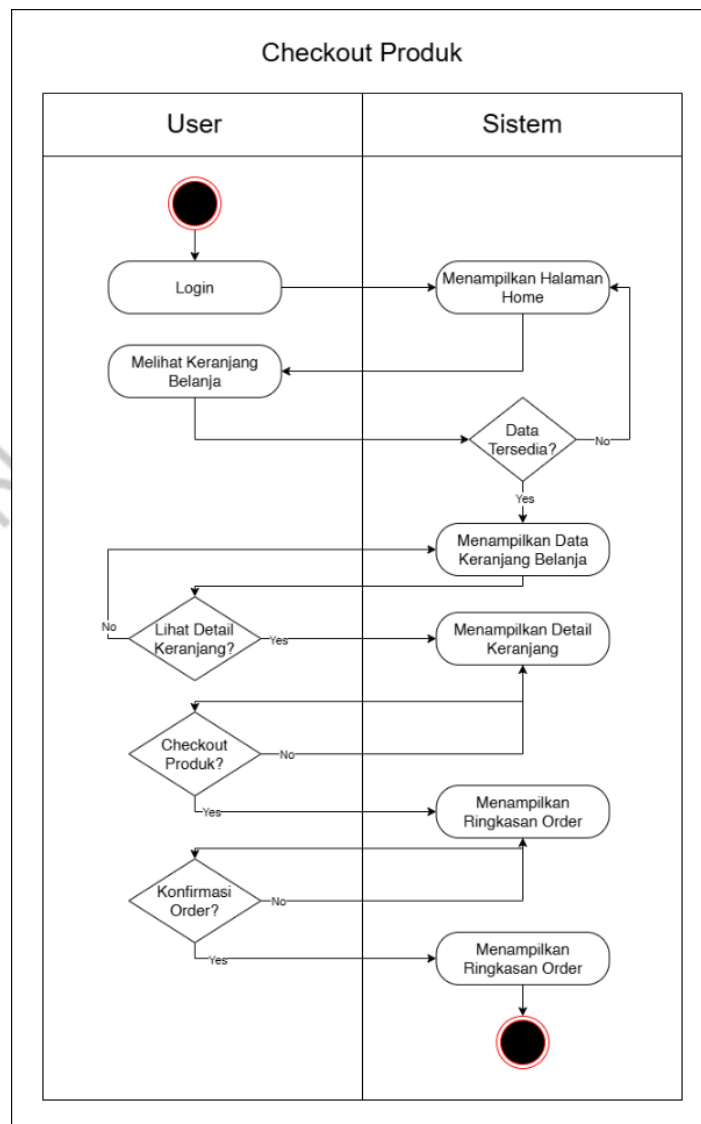
Diagram ini menunjukkan alur ketika pengguna memilih produk dan menembahkannya ke keranjang. Setelah login, pengguna mengakses katalog dan sistem menampilkan data produk yang tersedia. Jika pengguna memilih untuk melihat detail produk, sistem menampilkan informasi lengkap tersebut. Ketika pengguna menekan tombol untuk menambah produk ke keranjang, sistem

memproses permintaan tersebut dan menyimpan produk ke dalam keranjang belanja.



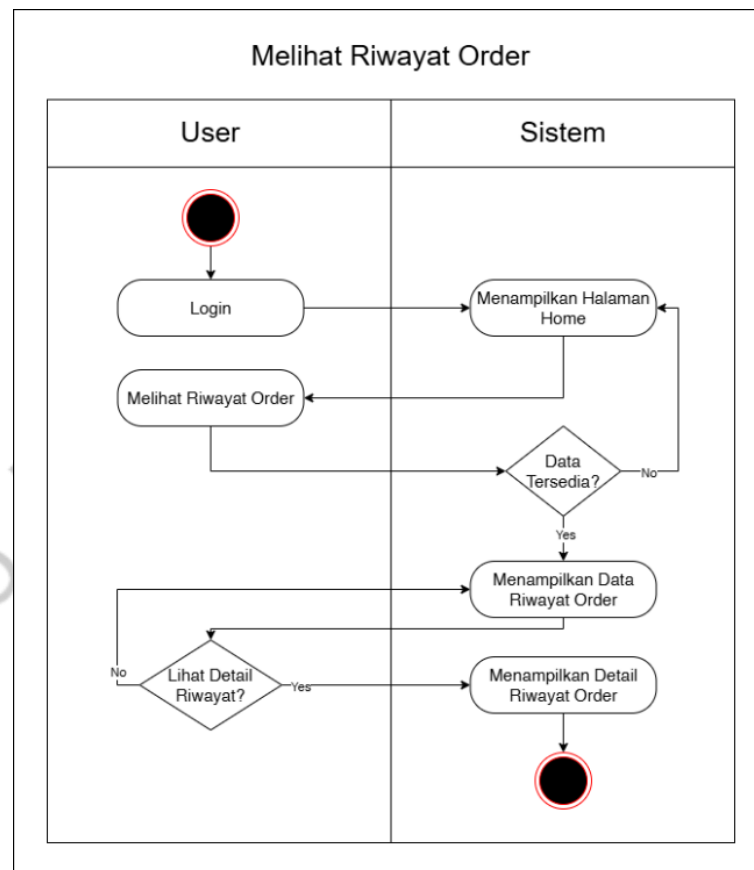
Gambar 3.12 Activity Diagram Melihat Keranjang Belanja

Diagram ini menjelaskan alur ketika pengguna ingin melihat isi keranjang belanja. Setelah login, pengguna memilih menu keranjang dan sistem memeriksa ketersediaan data. Jika data keranjang tersedia, sistem akan menampilkan daftar item yang ada. Pengguna juga dapat memilih untuk melihat detail dari produk tertentu, dan sistem menampilkan informasi detail tersebut sebelum proses berakhir.



Gambar 3.13 Activity Diagram Checkout Produk

Activity diagram ini menggambarkan alur proses checkout. Setelah pengguna mengakses keranjang dan data tersedia, sistem menampilkan isi keranjang. Jika pengguna melihat detail atau melanjutkan ke tahap checkout, sistem menampilkan ringkasan order. Pengguna kemudian diberikan opsi untuk mengonfirmasi pesanan. Jika dikonfirmasi, sistem menampilkan ringkasan akhir pesanan serta memproses transaksi.

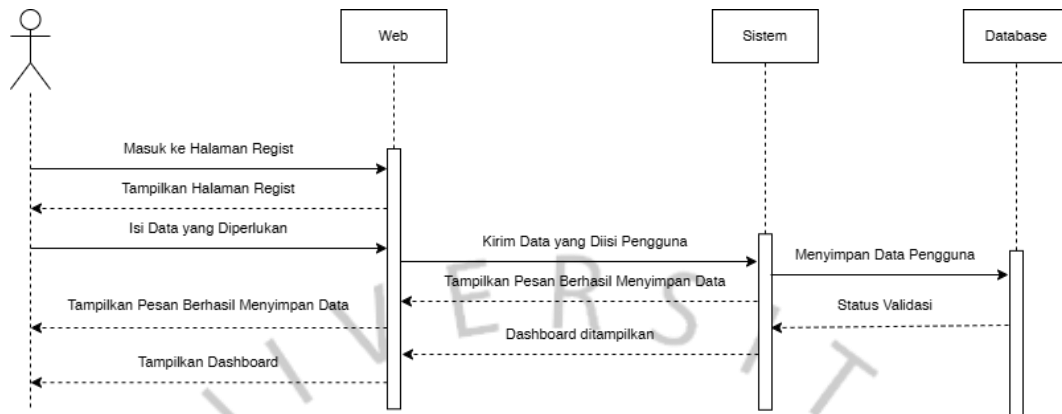


Gambar 3.14 Activity Diagram Melihat Riwayat Order

Diagram ini mendeskripsikan proses ketika pengguna ingin melihat riwayat transaksi yang pernah dilakukan. Setelah login dan masuk ke menu riwayat, sistem memeriksa apakah terdapat data riwayat. Jika tersedia, sistem menampilkan daftar riwayat order. Pengguna juga dapat memilih untuk melihat detail salah satu transaksi, dan sistem menampilkan informasi lengkap order tersebut sebelum proses berakhir

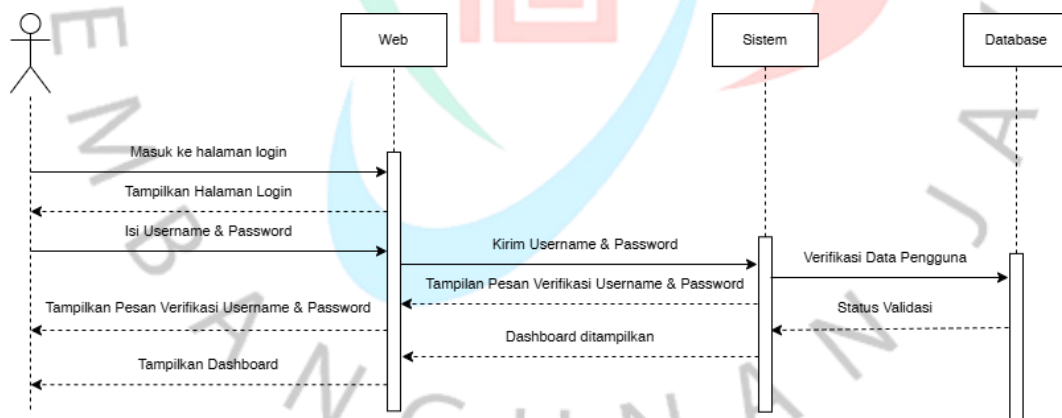
Selanjutnya, sequence Diagram di bawah ini digunakan untuk menggambarkan urutan proses yang terjadi ketika pengguna berinteraksi dengan sistem. Diagram ini menunjukkan bagaimana pesan dikirim antar komponen, mulai dari interface web, sistem, hingga database. Melalui sequence diagram, alur logika untuk setiap fitur seperti login, registrasi, pengelolaan keranjang, checkout, hingga

pengelolaan profil dapat dipahami dengan jelas sebelum diimplementasikan pada aplikasi.



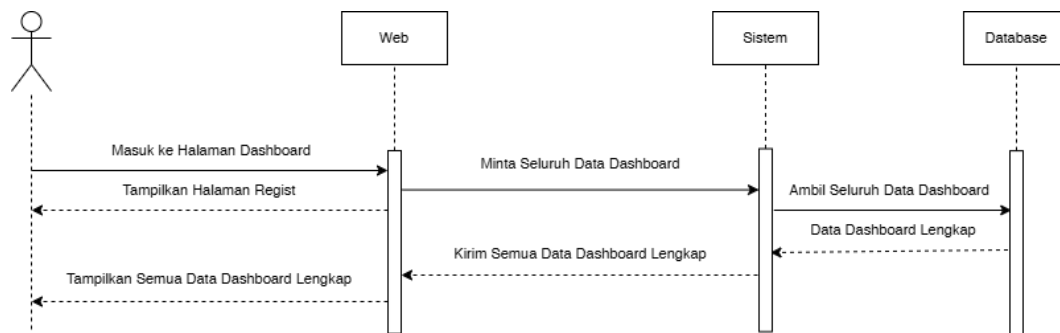
Gambar 3.2 Sequence Diagram Halaman Login

Sequence diagram ini menggambarkan alur ketika pengguna memasukkan email dan password pada halaman login. Sistem kemudian melakukan validasi kredensial ke database. Jika data sesuai, sistem mengembalikan respons berhasil dan pengguna diarahkan ke halaman dashboard. Jika tidak sesuai, sistem mengembalikan pesan kesalahan agar pengguna dapat mencoba kembali.



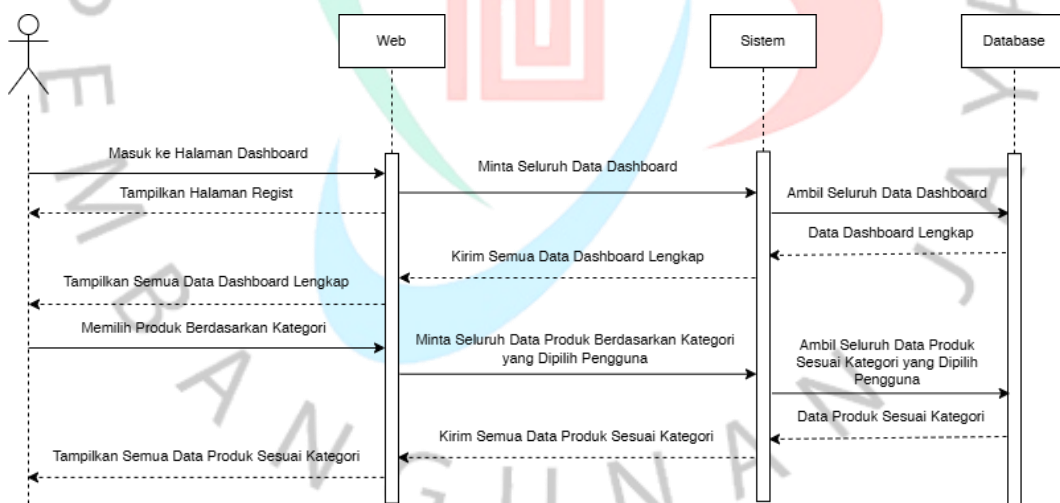
Gambar 3.3 Sequence Diagram Halaman Register

Diagram ini menjelaskan proses ketika pengguna mengisi formulir registrasi dan mengirimkan data ke sistem. Sistem melakukan validasi dan menyimpan data pengguna baru ke dalam database. Setelah berhasil disimpan, sistem memberikan konfirmasi dan pengguna diarahkan ke halaman login untuk melakukan autentikasi pertama kali.



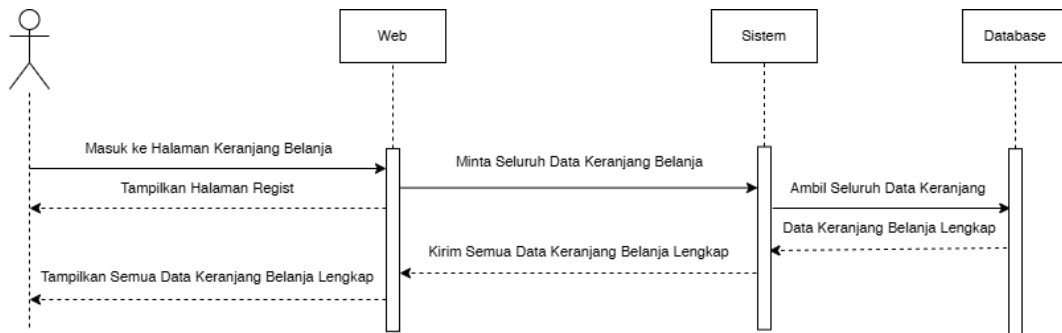
Gambar 3.4 Sequence Diagram Halaman Dashboard

Diagram ini menunjukkan alur ketika pengguna mengakses halaman dashboard atau halaman utama. Sistem mengambil data produk, kategori, dan rekomendasi dari database, kemudian mengirimkannya kembali ke web untuk ditampilkan kepada pengguna. Proses ini memastikan pengguna melihat informasi produk yang terbaru dan relevan.



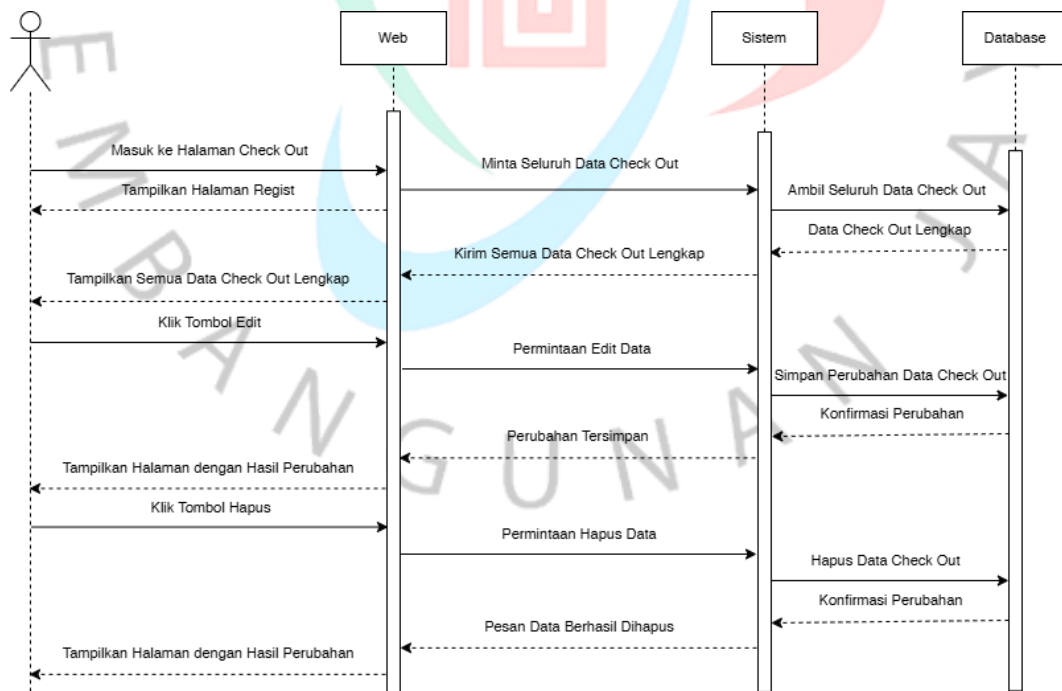
Gambar 3.5 Sequence Diagram Halaman Filtering

Sequence diagram ini menggambarkan proses ketika pengguna memilih kategori atau melakukan filter tertentu pada halaman produk. Sistem menerima permintaan filter, mengambil data produk yang sesuai dari database, lalu mengembalikan daftar produk yang telah difilter agar dapat ditampilkan kembali pada halaman web.



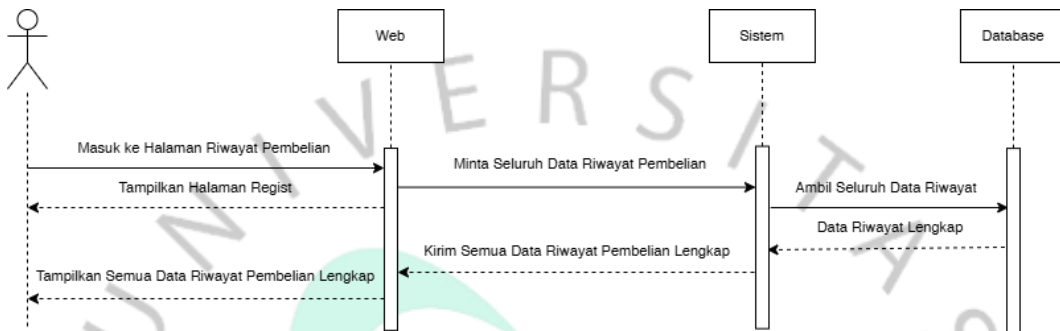
Gambar 3.6 Sequence Diagram Halaman Keranjang Belanja

Diagram ini memperlihatkan alur interaksi ketika pengguna menambahkan produk ke dalam keranjang. Web mengirim data produk dan jumlah ke sistem, kemudian sistem menyimpan atau memperbarui data keranjang pada database. Setelah berhasil, sistem mengembalikan status untuk menampilkan pembaruan keranjang ke pengguna.



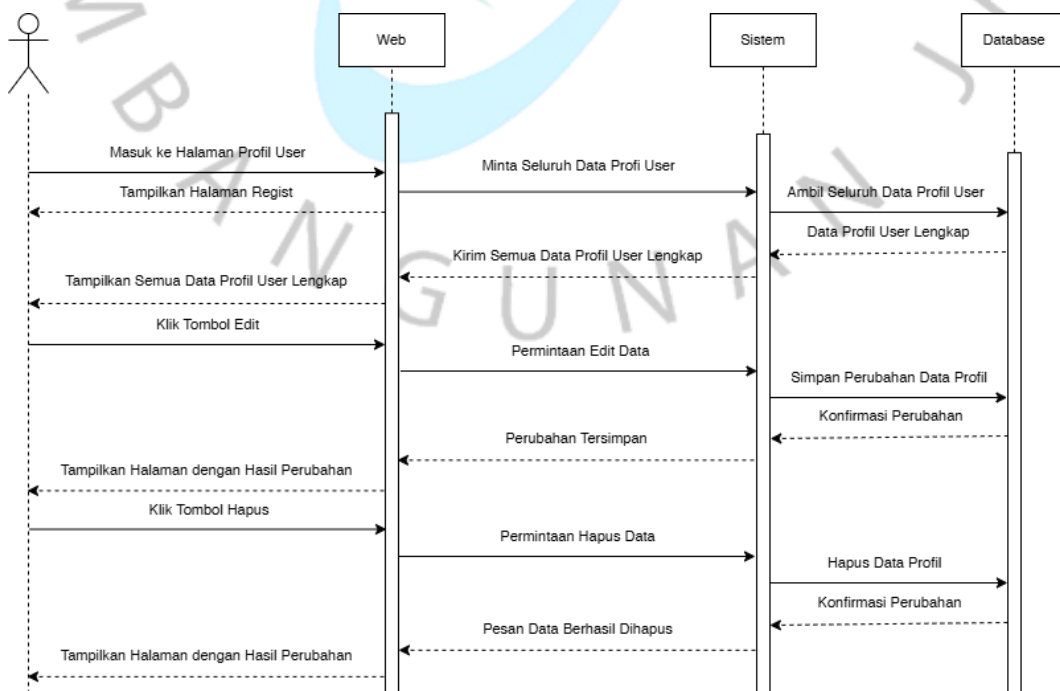
Gambar 3.7 Sequence Diagram Halaman Check Out Produk

Diagram checkout menjelaskan proses saat pengguna melanjutkan pembelian dari keranjang. Sistem memvalidasi data checkout, menyimpan detail transaksi ke tabel pesanan dan item pesanan, serta memperbarui stok produk bila diperlukan. Setelah semuanya tersimpan, sistem mengirimkan respons sukses dan menampilkan ringkasan pesanan kepada pengguna.



Gambar 3.8 Sequence Diagram Halaman Riwayat Pembelian

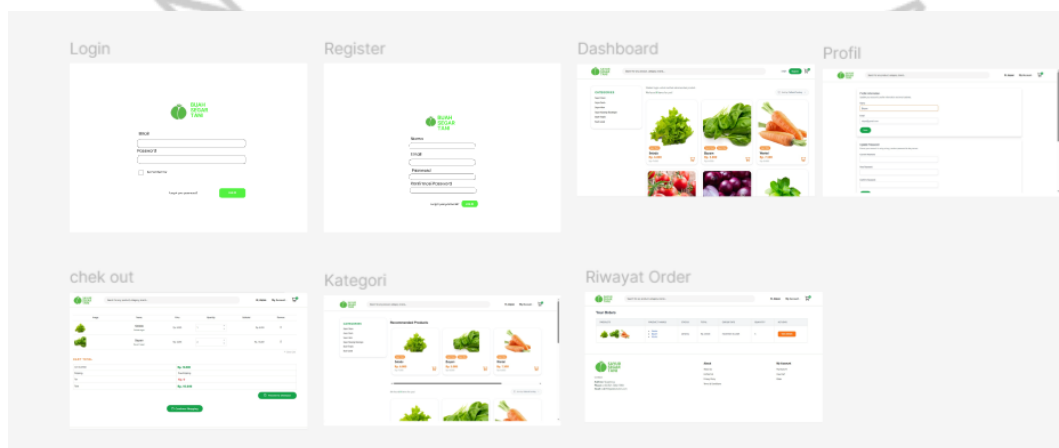
Sequence diagram ini menjelaskan alur ketika pengguna membuka halaman riwayat pesanan. Web mengirim permintaan ke sistem untuk mengambil seluruh pesanan berdasarkan ID pengguna. Sistem mengambil data pesanan dan detailnya dari database, lalu mengembalikannya dalam bentuk daftar riwayat yang siap ditampilkan di web.



Gambar 3.9 Sequence Diagram Halaman Profil Pengguna

Diagram ini menggambarkan alur saat pengguna mengakses atau memperbarui profil. Web mengambil data profil dari sistem, kemudian menampilkannya ke pengguna. Jika pengguna melakukan perubahan data, sistem memvalidasi input kemudian memperbarui informasi pengguna pada database dan mengembalikan status berhasil.

3.2.3 Perancangan Antarmuka Aplikasi



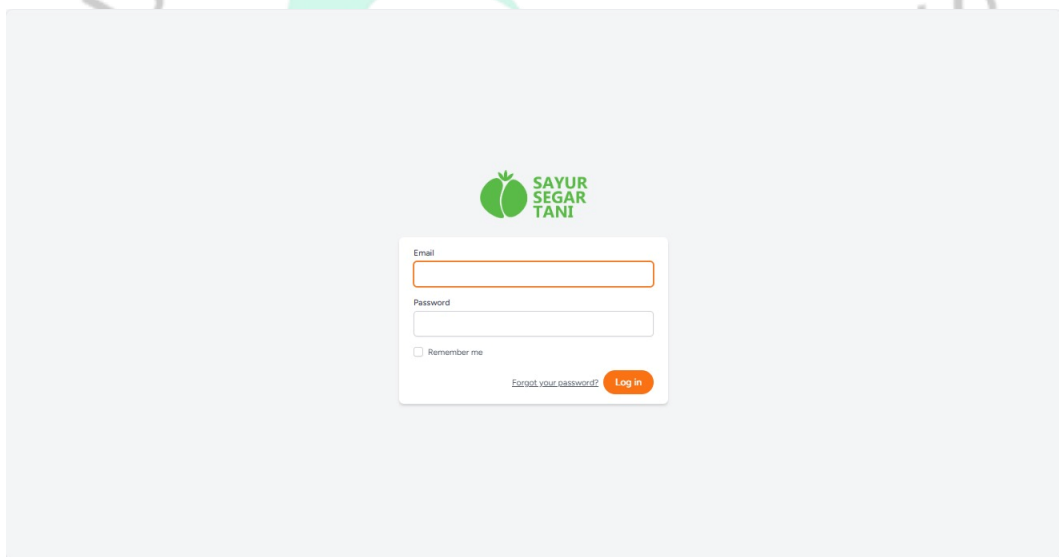
Gambar 3.22 Tampilan Mockup keseluruhan website puregreen

Perancangan Antarmuka Aplikasi merupakan tahapan pembuatan desain tampilan website PureGreens berdasarkan hasil analisis kebutuhan pengguna dan konsep desain yang telah ditetapkan. Pada tahap ini, setiap halaman dirancang dengan memperhatikan konsistensi antarmuka, responsivitas, serta pengalaman pengguna (User Experience/UX) secara optimal. Desain antarmuka dibuat untuk mendukung seluruh aktivitas pengguna, mulai dari proses autentikasi, penelusuran produk, pengelolaan keranjang belanja, hingga peninjauan riwayat pesanan.

Jenis perancangan antarmuka yang digunakan adalah mockup high-fidelity, yaitu representasi visual dengan tingkat detail tinggi yang menggambarkan tampilan akhir aplikasi secara realistis. Mockup high-fidelity memungkinkan pengembang dan pemangku kepentingan melakukan evaluasi visual terhadap tata letak, tipografi, ikon, warna, serta interaksi yang akan digunakan dalam aplikasi

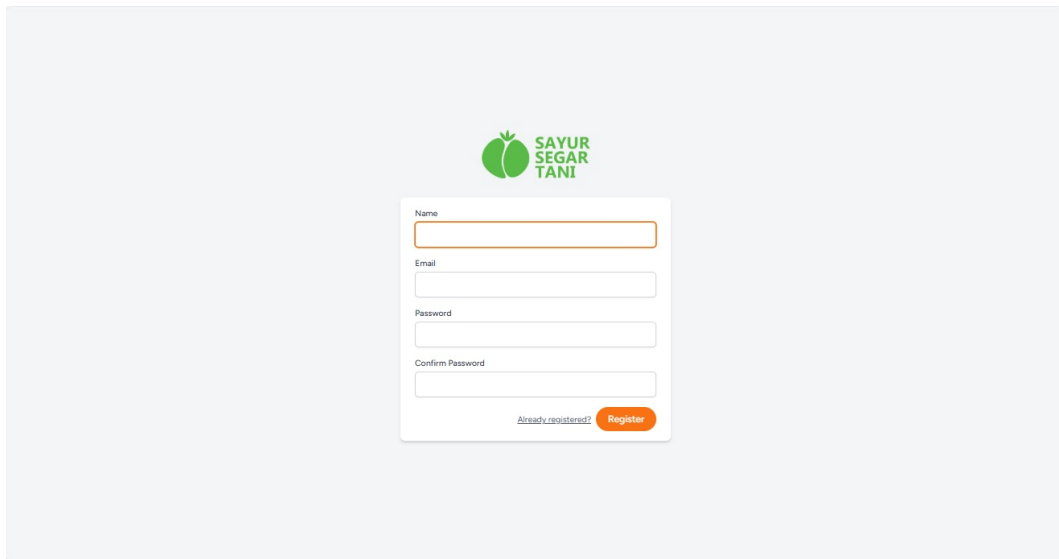
sebelum proses implementasi kode dilakukan. Proses perancangan dilakukan menggunakan Figma sebagai tools desain antarmuka karena mendukung kolaborasi, pengaturan komponen desain secara sistematis, dan pembuatan prototipe interaktif secara cepat.

Hasil perancangan mockup terdiri dari beberapa halaman utama, antara lain halaman login, register, dashboard pengguna, katalog produk, halaman checkout, serta riwayat pemesanan. Setiap komponen visual didesain berdasarkan prinsip desain UI/UX, seperti hierarki visual, konsistensi elemen, kemudahan navigasi, dan desain berbasis pengguna (user-centered design), agar pengalaman pengguna lebih efektif dan mudah dipahami. Gambar 3.22 memperlihatkan tampilan mockup keseluruhan website PureGreens yang telah dirancang menggunakan Figma



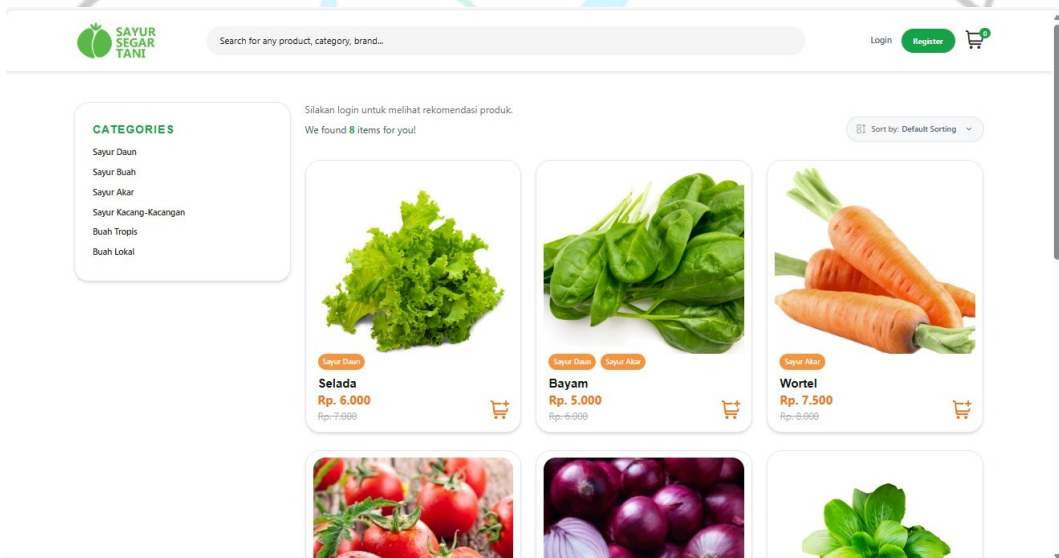
Gambar 3.15 Tampilan Halaman Login

Halaman login digunakan sebagai akses awal bagi pengguna untuk masuk ke dalam sistem PureGreens. Halaman ini menyediakan form untuk memasukkan email dan password, serta opsi “Remember me” dan tautan untuk melakukan reset password. Tampilan dirancang sederhana dan terpusat agar mudah diakses serta memastikan proses autentikasi berjalan dengan jelas dan mudah dipahami oleh pengguna.



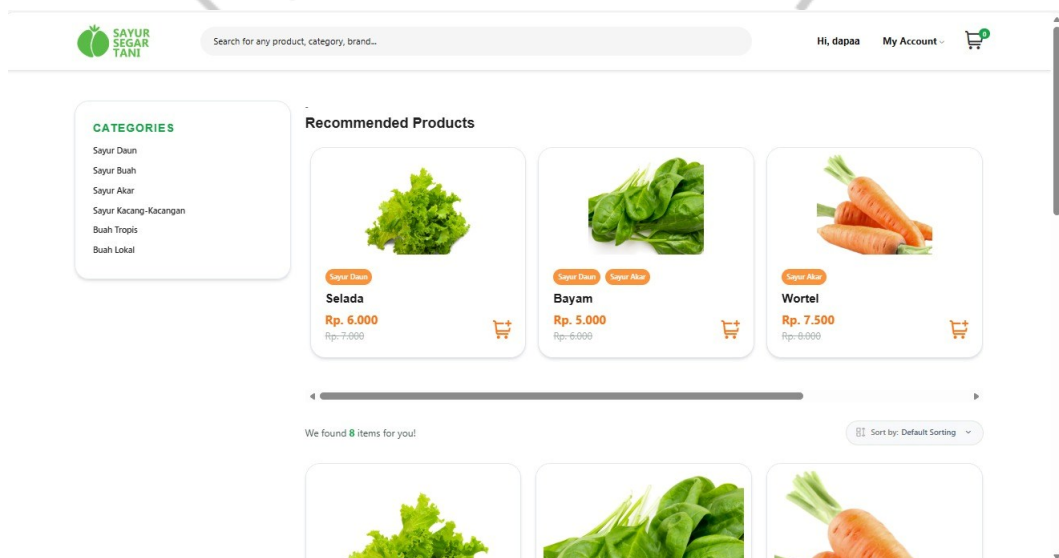
Gambar 3.16 Tampilan Halaman Regist

Halaman registrasi digunakan oleh pengguna baru untuk membuat akun pada website PureGreens. Form yang tersedia mencakup input nama, email, password, dan konfirmasi password. Tampilan dirancang minimalis dan terpusat untuk memudahkan pengguna dalam mengisi data dengan cepat. Tombol “Register” disediakan untuk mengirimkan data ke sistem, sedangkan tautan “Already registered?” mengarahkan pengguna ke halaman login.



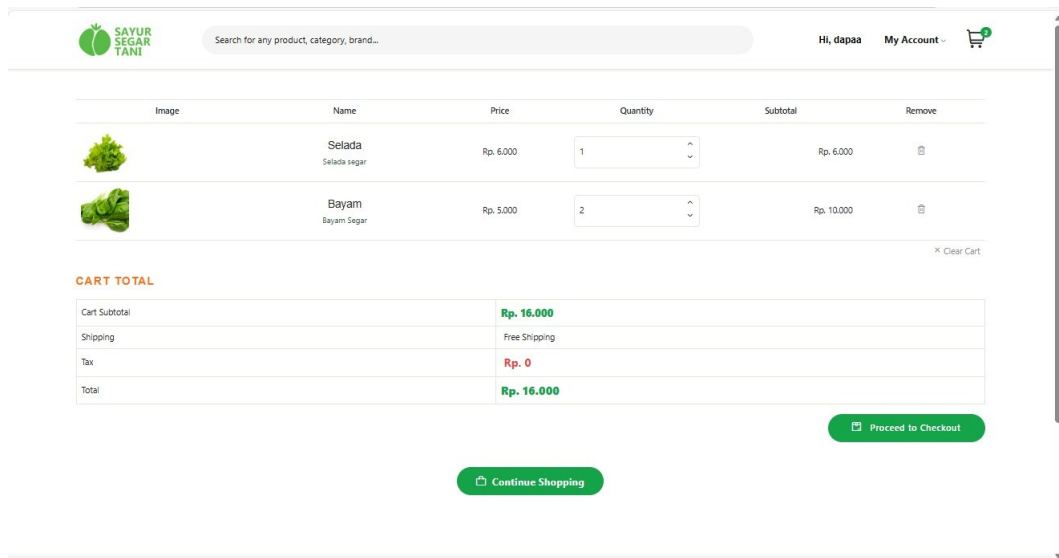
Gambar 3.17 Tampilan Halaman Home

Halaman Home menampilkan daftar produk utama yang tersedia pada website PureGreens. Pada bagian kiri terdapat menu kategori untuk memudahkan pengguna dalam menelusuri jenis produk seperti sayur daun, sayur akar, buah lokal, dan lainnya. Bagian utama halaman menampilkan card produk lengkap dengan gambar, nama, kategori, harga, serta tombol untuk menambahkan item ke keranjang. Selain itu, tersedia juga fitur pengurutan produk melalui dropdown “Sort by” agar pengguna dapat menampilkan produk sesuai kebutuhan. Tampilan halaman dirancang bersih dan informatif sehingga memudahkan pengguna dalam melihat seluruh produk yang ditawarkan.



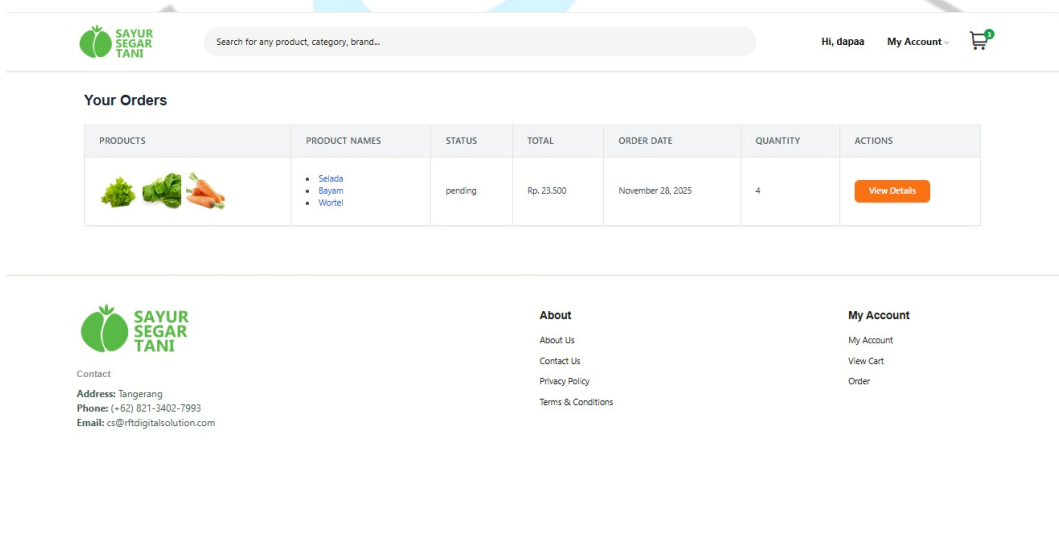
Gambar 3.18 Tampilan Halaman Rekomendasi

Halaman rekomendasi menampilkan daftar produk yang direkomendasikan untuk pengguna. Produk ditampilkan dalam bentuk card berisi gambar, nama, kategori singkat, dan harga, serta tombol untuk menambahkan produk ke keranjang. Pada bagian kiri tetap disediakan menu kategori, sehingga pengguna dapat berpindah antar jenis produk dengan mudah. Halaman ini membantu pengguna menemukan produk utama secara lebih terarah tanpa harus menelusuri seluruh katalog terlebih dahulu.



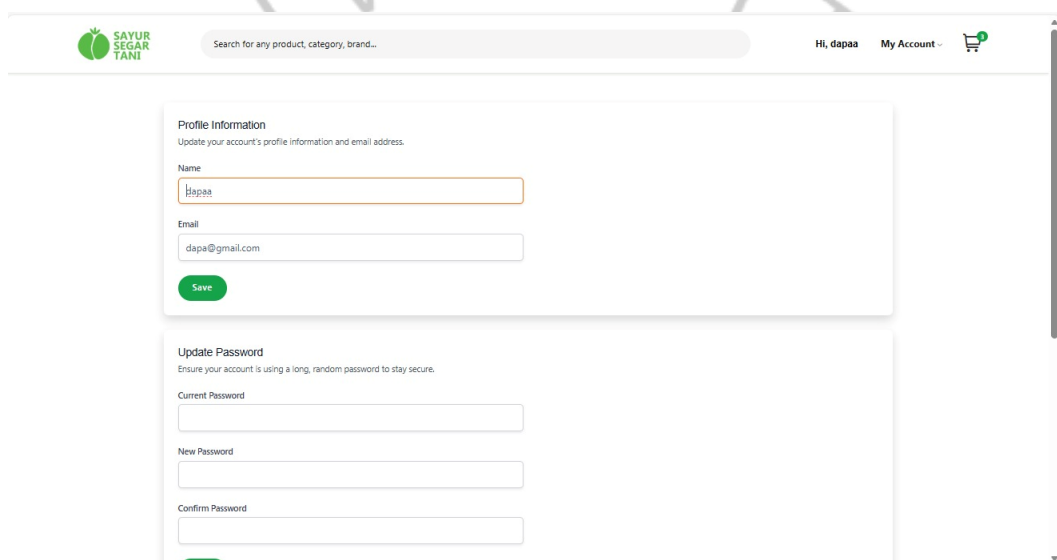
Gambar 3.19 Tampilan Halaman CheckOut

Halaman checkout menampilkan ringkasan belanja pengguna sebelum melanjutkan proses pemesanan. Pada bagian atas terdapat daftar produk dalam keranjang beserta gambar, harga satuan, jumlah item, dan subtotal harga. Pengguna dapat mengubah jumlah atau menghapus produk langsung dari halaman ini. Bagian “Cart Total” menampilkan perhitungan akhir seperti subtotal, biaya pengiriman, pajak, dan total keseluruhan. Tombol “Proceed to Checkout” disediakan sebagai langkah lanjut untuk menyelesaikan pesanan, sedangkan tombol “Continue Shopping” mengarahkan pengguna kembali ke halaman produk.



Gambar 3.20 Tampilan Halaman Riwayat Belanja

Halaman riwayat pesanan menampilkan daftar transaksi yang pernah dilakukan oleh pengguna. Setiap riwayat berisi informasi produk yang dipesan, status pesanan, total pembayaran, tanggal pemesanan, serta jumlah item. Terdapat pula tombol “View Details” yang memungkinkan pengguna melihat informasi pesanan secara lebih lengkap. Halaman ini membantu pengguna memantau pesanan sebelumnya dan memastikan proses transaksi berjalan dengan baik.



The screenshot displays the user profile management interface. At the top, the 'SAYUR SEGAR TANI' logo is visible on the left, and a search bar, user greeting 'Hi, dapa', 'My Account' link, and shopping cart icon are on the right. The main content area contains two forms. The first form, titled 'Profile Information', prompts the user to 'Update your account's profile information and email address.' It includes a 'Name' field with the value 'Hapaa' and an 'Email' field with 'dapa@gmail.com'. A green 'Save' button is positioned below these fields. The second form, titled 'Update Password', prompts the user to 'Ensure your account is using a long, random password to stay secure.' It features three input fields: 'Current Password', 'New Password', and 'Confirm Password'.

Gambar 3.21 Tampilan Halaman Profil Pengguna

Halaman profil pengguna memungkinkan user untuk memperbarui informasi akun seperti nama dan email. Selain itu, tersedia juga bagian untuk mengganti password dengan memasukkan password lama, password baru, dan konfirmasi password. Tampilan halaman dibuat sederhana dan terstruktur agar proses pembaruan data dapat dilakukan dengan mudah dan aman oleh pengguna.

3.2.4 Implementasi Program *Frontend*

Tahap ini merupakan proses penerapan rancangan antarmuka website PureGreens ke dalam bentuk kode menggunakan framework Laravel. Implementasi difokuskan pada pembuatan halaman-halaman utama seperti autentikasi pengguna (login dan register), dashboard produk, kategori, checkout, serta riwayat pesanan. File utama yang digunakan adalah index.blade.php

sebagai halaman beranda, yang terhubung dengan basis data untuk menampilkan informasi produk dan konten lainnya secara dinamis. Proses implementasi ini memanfaatkan Blade Template Laravel dan pengaturan gaya menggunakan TailwindCSS agar tampilan website responsif, konsisten, dan mudah digunakan oleh pengguna pada berbagai perangkat.

```

<!-- Guest Layout -->
<!-- Session Status -->
<x-auth-session-status class="mb-4" :status="session('status')"/>

<form method="POST" action="{{ route('login') }}">
    @csrf

    <!-- Email Address -->
    <div>
        <x-input-label for="email" :value="__('Email')"/>
        <x-text-input id="email" class="block w-full" type="email" name="email" :value="old('email')"/> required autofocus autocomplete="username"/>
        <x-input-error :messages="$errors->get('email')"/> class="mt-2"/>
    </div>

    <!-- Password -->
    <div class="mt-4">
        <x-input-label for="password" :value="__('Password')"/>
        <x-text-input id="password" class="block w-full" type="password" name="password" :value="old('password')"/> required autocomplete="current-password"/>
        <x-input-error :messages="$errors->get('password')"/> class="mt-2"/>
    </div>

    <!-- Remember Me -->
    <div class="block mt-4">
        <label for="remember_me" class="inline-flex items-center">
            <input id="remember_me" type="checkbox" class="rounded poorbg-gray-900 border-gray-300 poorborder-gray-700 text-orange-600 shadow-sm focus:ring-orange-500 poorfocus:ring-orange-600"/>
            <span class="ml-2 text-sm text-gray-600 poor-text-gray-400">{{ __('Remember me') }}</span>
        </label>
    </div>

    <div class="flex items-center justify-end mt-4">
        @if (Route::has('password.request'))
            <a class="underline text-sm text-gray-600 poor-text-gray-400 hover:text-gray-900 poorhover:text-gray-100 rounded-md focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-orange-500 poorfocus:ring-orange-600" href="{{ route('password.request') }}">
                {{ __('Forgot your password?') }}
            </a>
        @endif

        <x-primary-button class="bg-orange-500 px-5 py-2 text-white font-medium rounded-3xl ml-2">
            {{ __('Log in') }}
        </x-primary-button>
    </div>
</form>

```

Gambar 3.23 Program Halaman Login

Kode di atas merupakan bagian dari halaman **Login** Laravel Breeze yang membangun form autentikasi menggunakan Blade Components dan Tailwind CSS. Form ini mengirim data melalui metode POST ke route `login` dan dilindungi dengan `@csrf`. Komponen seperti `<x-input-label>`, `<x-text-input>`, dan `<x-input-error>` digunakan untuk menampilkan input email dan password beserta pesan validasinya. Tersedia pula opsi "Remember me" melalui checkbox, serta link "Forgot your password?" yang ditampilkan jika route tersedia. Tombol login dibuat menggunakan `<x-primary-button>`. Seluruh kode ini berfungsi untuk menampilkan UI form login secara rapi dan responsif tanpa menangani proses autentikasi secara langsung.

```

@guest-layout
<form method="POST" action="{{ route('register') }}">
    @csrf

    <!-- Name -->
    <div>
        <x-input-label for="name" :value="__('Name')"/>
        <x-text-input id="name" class="block mt-1 w-full" type="text" name="name" :value="old('name')" required autofocus autocomplete="name" />
        <x-input-error :messages="$errors->get('name')" class="mt-2" />
    </div>

    <!-- Email Address -->
    <div class="mt-4">
        <x-input-label for="email" :value="__('Email')"/>
        <x-text-input id="email" class="block mt-1 w-full" type="email" name="email" :value="old('email')" required autocomplete="username" />
        <x-input-error :messages="$errors->get('email')" class="mt-2" />
    </div>

    <!-- Password -->
    <div class="mt-4">
        <x-input-label for="password" :value="__('Password')"/>
        <x-text-input id="password" class="block mt-1 w-full" type="password" name="password" required autocomplete="new-password" />
        <x-input-error :messages="$errors->get('password')" class="mt-2" />
    </div>

    <!-- Confirm Password -->
    <div class="mt-4">
        <x-input-label for="password_confirmation" :value="__('Confirm Password')"/>
        <x-text-input id="password_confirmation" class="block mt-1 w-full" type="password" name="password_confirmation" required autocomplete="new-password" />
        <x-input-error :messages="$errors->get('password_confirmation')" class="mt-2" />
    </div>

    <div class="flex items-center justify-end mt-4">
        <a class="underline text-sm text-gray-600 hover:text-gray-900 poorhover:text-gray-100 rounded-md focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-indigo-500" href="#">
            {{ __('Already registered?') }}
        </a>

        <x-primary-button class="bg-orange-500 px-5 py-2 text-white font-medium rounded-3xl ml-2">
            {{ __('Register') }}
        </x-primary-button>
    </div>
</form>
</guest-layout>

```

Gambar 3.24 Program Halaman register

Kode di atas adalah tampilan form pendaftaran (Register) pada Laravel Breeze yang menggunakan Blade Components dan Tailwind CSS untuk membangun UI. Form dikirim melalui metode POST ke route register dan dilindungi oleh @csrf. Komponen <x-input-label>, <x-text-input>, dan <x-input-error> digunakan untuk menampilkan input nama, email, password, dan konfirmasi password beserta validasinya. Semua input memiliki atribut required, dan password dilengkapi autocomplete khusus. Pada bagian bawah terdapat link menuju halaman login bagi pengguna yang sudah terdaftar, serta tombol <x-primary-button> untuk melakukan registrasi. Keseluruhan kode ini menangani tampilan form register tanpa memproses logika backend pendaftarannya.

```

1 <x-app-layout>
2 <div class="container mx-auto px-4 py-8">
3 <h1 class="text-2xl font-bold mb-6 text-gray-800">Your Orders</h1>
4
5 <div class="overflow-x-auto">
6 <table class="min-w-full bg-white border border-gray-200 rounded-lg shadow-sm">
7 <thead>
8 <tr class="bg-gray-100 text-left text-sm font-semibold text-gray-600 uppercase tracking-wider">
9 <th class="px-6 py-3">Products</th>
10 <th class="px-6 py-3">Product Names</th>
11 <th class="px-6 py-3">Status</th>
12 <th class="px-6 py-3">Total</th>
13 <th class="px-6 py-3">Order Date</th>
14 <th class="px-6 py-3">Quantity</th>
15 <th class="px-6 py-3">Actions</th>
16 </tr>
17 </thead>
18 <tbody>
19 @foreach ($orders as $order)
20 @php
21 $quantity = $order->orderItems->sum('quantity');
22 @endphp
23 <tr class="border-b border-gray-50 transition-colors duration-200">
24 <td class="px-6 py-4">
25 <div class="flex flex-wrap gap-2 items-center">
26 @foreach ($order->orderItems as $item)
27 <a href="{{ route('product.details', $item->product->id) }}">
28 product->name }}"
30 class="w-16 h-16 object-cover rounded-md">
31 </a>
32 @endforeach
33 </div>
34 <td class="px-6 py-4">
35 <ul class="list-disc list-inside text-sm text-gray-700">
36 @foreach ($order->orderItems as $item)
37 <li>
38 <a href="{{ route('product.details', $item->product->id) }}">
39 class="text-blue-600 hover:underline">
40 {{ $item->product->name }}
41 </a>
42 </li>
43 @endforeach
44 </ul>
45 </td>
46 <td class="px-6 py-4 text-sm text-gray-700">{{ $order->status }}</td>
47 <td class="px-6 py-4 text-sm text-gray-700">Rp. {{ number_format($order->total, 0, ',', '.') }}</td>
48 <td class="px-6 py-4 text-sm text-gray-700">{{ $order->created_at->format('F j, Y') }}</td>
49 <td class="px-6 py-4 text-sm text-gray-700">{{ $quantity }}</td>
50 <td class="px-6 py-4">
51 <a href="{{ route('invoice', $order) }}"
52 class="inline-block bg-orange-500 text-white px-4 py-2 text-sm font-semibold rounded-lg hover:bg-orange-600 transition-colors duration-200 shadow-sm">
53 View Details
54 </a>
55 </td>
56 </tr>
57 </tbody>
58 </table>
59 </div>
60 </div>
61 </x-app-layout>

```

Gambar 3.25 Program Halaman home

Kode di atas menampilkan halaman home dalam aplikasi Laravel yang menggunakan Blade template dan Tailwind CSS. Data pesanan dikirim melalui variabel \$orders, kemudian dilakukan loop @foreach untuk menampilkan setiap order dalam bentuk tabel. Pada setiap baris tabel, kode menghitung total jumlah item menggunakan \$order->orderItems->sum('quantity'), lalu menampilkan daftar gambar produk dan nama produk melalui loop kedua pada relasi orderItems. Setiap kolom tabel menampilkan status order, total harga dalam format rupiah, tanggal order dengan format tertentu, dan total quantity. Di kolom terakhir terdapat tombol link untuk melihat detail invoice menggunakan route invoice. Seluruh kode ini berfungsi untuk menampilkan data pesanan pengguna secara dinamis dan terstruktur di UI tanpa menangani logika pemrosesan data.

```

1  <?php
2
3  namespace App\Filament\Resources\CategoryResource\RelationManagers;
4
5  use Closure;
6  use Filament\Forms;
7  use Filament\Resources\Form;
8  use Filament\Resources\RelationManagers\RelationManager;
9  use Filament\Resources\Table;
10 use Filament\Tables;
11 use Illuminate\Database\Eloquent\Builder;
12 use Illuminate\Database\Eloquent\SoftDeletingScope;
13
14 class CategoriesRelationManager extends RelationManager
15 {
16     protected static string $relationship = 'categories';
17
18     protected static ?string $recordTitleAttribute = 'name';
19
20     public static function form(Form $form): Form
21     {
22         return $form
23             ->schema([
24                 Forms\Components\TextInput::make('name')
25                     ->reactive()
26                     ->afterStateUpdated(function (\Closure $set, $state) {
27                         $set('slug', \Illuminate\Support\Str::upper(\Illuminate\Support\Str::slug($state)));
28                     })
29                     ->dehydrateStateUsing(fn($state) => \Illuminate\Support\Str::ucfirst($state))
30                     ->autofocus()
31                     ->unique(ignoreRecord: true)
32                     ->required(),
33                 Forms\Components\TextInput::make('slug')
34                     ->disabled()
35                     ->required()
36                     ->dehydrateStateUsing(fn($state) => \Illuminate\Support\Str::upper($state)),
37             ]);
38     }
39
40
41
42     public static function table(Table $table): Table
43     {
44         return $table
45             ->columns([
46                 Tables\Columns\TextColumn::make('name'),
47                 Tables\Columns\TextColumn::make('slug'),
48             ])
49             ->filters([
50                 //
51             ])
52             ->headerActions([
53                 Tables\Actions\CreateAction::make(),
54             ])
55             ->actions([
56                 Tables\Actions\EditAction::make(),
57                 Tables\Actions\DeleteAction::make(),
58             ])
59             ->bulkActions([
60                 Tables\Actions\DeleteBulkAction::make(),
61             ]);
62     }
63 }
64

```

Gambar 3.26 Program Halaman rekomendasi

Kode di atas adalah Relation Manager Filament untuk mengelola relasi kategori dalam sebuah resource Laravel. Class CategoriesRelationManager memperluas RelationManager dan menentukan relasi yang dikelola melalui properti `$relationship = 'categories'` serta nama atribut yang ditampilkan pada record `$recordTitleAttribute = 'name'`.

menggunakan `$recordTitleAttribute = 'name'`. Method `form()` membangun form input dengan komponen `TextInput` untuk field name dan slug, di mana name bersifat reactive dan setiap perubahan akan otomatis menghasilkan slug menggunakan fungsi `afterStateUpdated`. Nilai slug di-generate dari nama dengan format huruf besar dan unik untuk setiap record. Field slug juga dibuat disabled agar tidak diubah manual. Method `table()` membangun tampilan tabel yang menampilkan kolom name dan slug lengkap dengan aksi create, edit, delete, serta delete bulk. Kode ini berfungsi untuk membuat antarmuka CRUD relasi kategori secara cepat di panel admin Filament tanpa menulis banyak kode manual.

```

1  </php>
2
3  namespace App\Http\Livewire;
4
5  use App\Mail\OrderReceived;
6  use App\Models\Order;
7  use App\Models\Product;
8  use App\Services\InvoiceService;
9  use Livewire\Component;
10 use App\Http\Livewire\Cart;
11 use Illuminate\Http\Request;
12 use Illuminate\Support\Facades\Auth;
13 use Illuminate\Support\Facades\Http;
14 use Illuminate\Support\Facades\Log;
15 use Illuminate\Support\Facades\Session;
16 use Symfony\Component\HttpFoundation\Exception\NotFoundHttpException;
17
18 class Checkout extends Component
19 {
20     public function initiatePayment()
21     {
22         $user = Auth::user();
23         $cart = session()->get('cart', []);
24
25         if (empty($cart)) {
26             session()->flash('error', 'Keranjang kosong');
27             return redirect()->route('cart');
28         }
29
30         $products = [];
31         $prices = [];
32         $quantities = [];
33         $descriptions = [];
34         $totalAmount = 0;
35
36         foreach ($cart as $item) {
37             // Pastikan struktur data cart konsisten
38             $product = $item['model'] ?? $item['product'] ?? null;
39             $qty = (int) ($item['qty'] ?? $item['quantity'] ?? 1);
40
41             if (!$product) {
42                 Log::error('Product not found in cart item: ', $item);
43                 continue;
44             }
45
46             $products[] = $product->name;
47             $prices[] = (int) $product->price; // Pastikan integer
48             $quantities[] = $qty;
49             $descriptions[] = $product->description ?? 'Produk ' . $product->name;
50             $totalAmount += $product->price * $qty;
51         }
52
53         // Validasi apakah ada produk
54         if (empty($products)) {
55             session()->flash('error', 'Tidak ada produk valid dalam keranjang');
56             return redirect()->route('cart');
57         }
58
59         // Generate unique transaction ID
60         $trid = 'TRX-' . time() . '-' . uniqid();
61
62         // Simpan transaksi ke database
63         $order = Order::create([
64             'user_id' => $user->id,
65             'total' => $totalAmount,
66             'status' => 'pending',
67             'tr_id' => $trid,
68         ]);
69
70         // Simpan order items
71         foreach ($cart as $item) {
72             $product = $item['model'] ?? $item['product'] ?? null;
73             $qty = (int) ($item['qty'] ?? $item['quantity'] ?? 1);
74
75             if ($product) {
76                 OrderItem::create([
77                     'order_id' => $order->id,
78                     'product_id' => $product->id,
79                     'quantity' => $qty,
80                     'price' => $product->price,
81                 ]);
82             }
83         }
84     }
85 }

```

Gambar 3.27 Program Halaman checkout

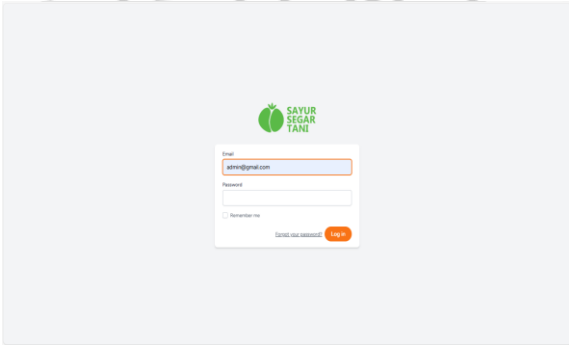
Kode tersebut merupakan fungsi `initiatePayment()` dalam komponen `Checkout` Livewire yang menangani proses awal transaksi berdasarkan data keranjang

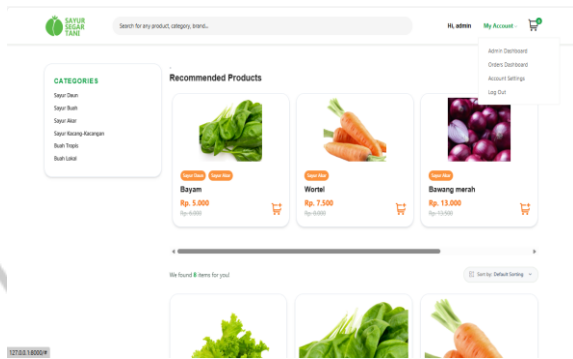
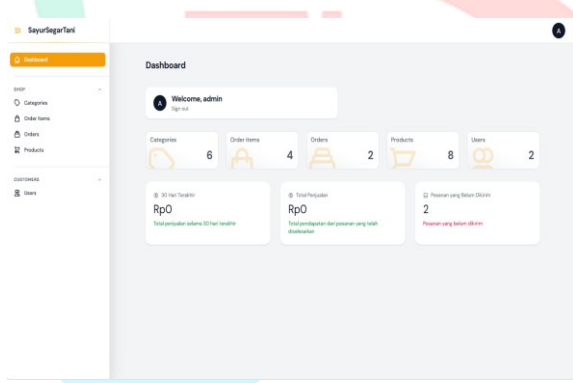
yang disimpan di session. Pertama, fungsi mengambil user yang login dan isi cart; jika kosong, pengguna diarahkan kembali dengan pesan error. Fungsi kemudian melakukan iterasi pada setiap item cart untuk memastikan struktur datanya valid, mengambil model produk, menghitung jumlah dan total harga, serta mencatat informasi produk yang berhasil diproses, sambil mengabaikan item yang tidak valid dan mencatatnya pada log. Setelah semua produk diverifikasi, sistem membuat ID transaksi unik (TRX-...), menyimpan data transaksi ke tabel orders, lalu menyimpan setiap item cart sebagai detail pesanan di tabel orderItems. Keseluruhan proses ini memastikan bahwa data keranjang diverifikasi, total dihitung, dan transaksi beserta detail produknya tersimpan dengan benar sebelum melanjutkan ke tahap pembayaran.

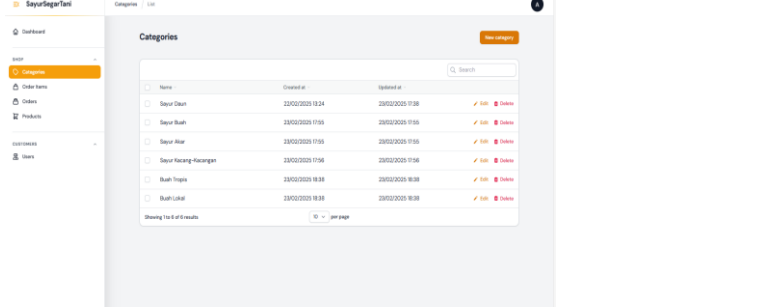
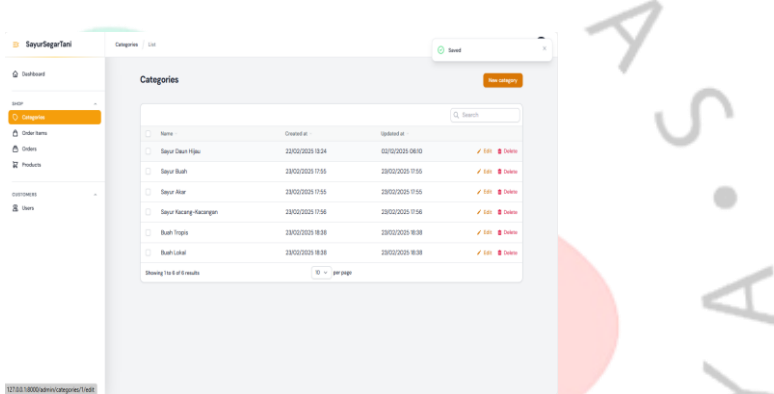
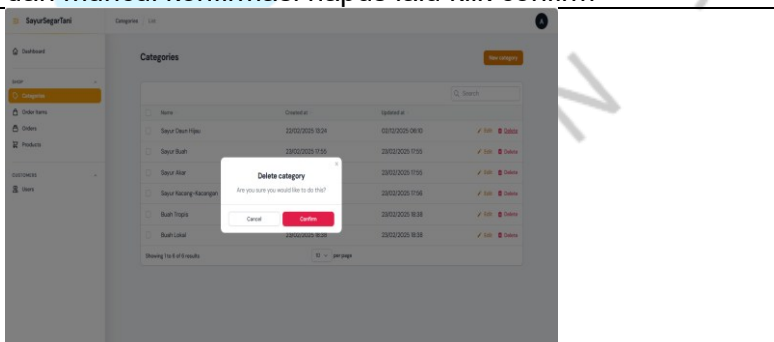
3.2.5 Pengujian Aplikasi

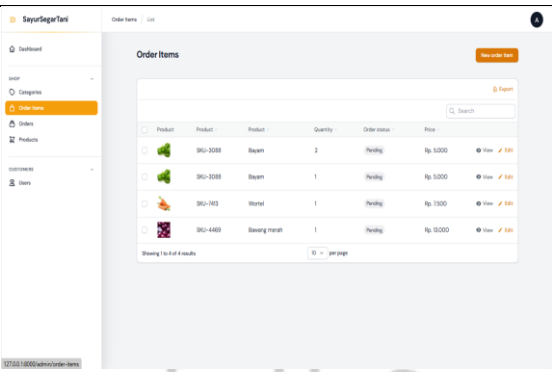
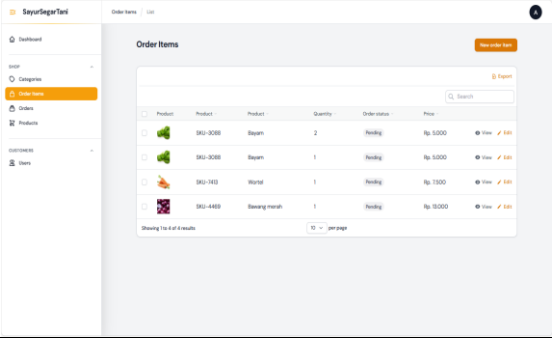
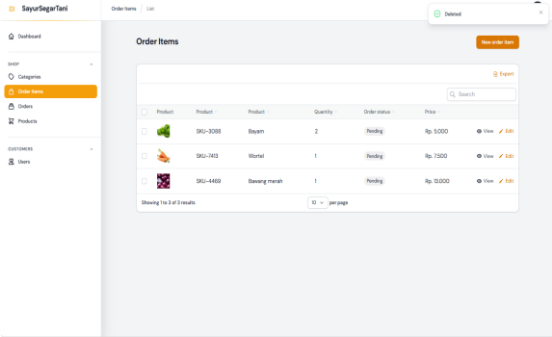
Tahap pengujian aplikasi dilakukan untuk memastikan bahwa setiap fitur pada sistem e-commerce bekerja sesuai dengan kebutuhan serta pola penggunaan yang diharapkan dari sisi pengguna. Metode pengujian yang digunakan adalah black-box testing, yaitu pengujian yang berfokus pada fungsi aplikasi berdasarkan input dan output tanpa melihat struktur internal kode (Abdillah et al., 2023). Pendekatan ini dipilih karena sesuai dengan ruang lingkup kerja profesi yang berfokus pada tampilan antarmuka, alur interaksi pengguna, serta kejelasan navigasi di setiap halaman.

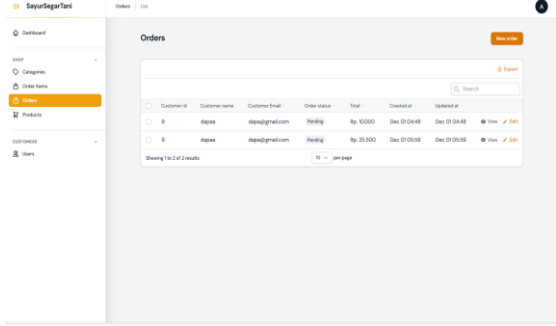
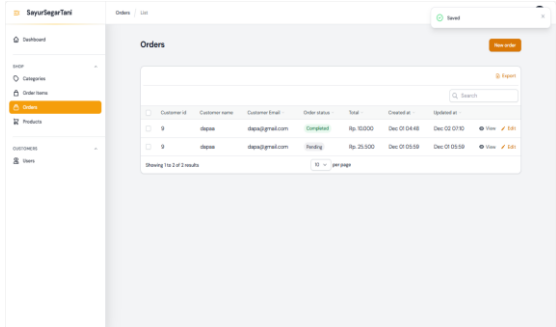
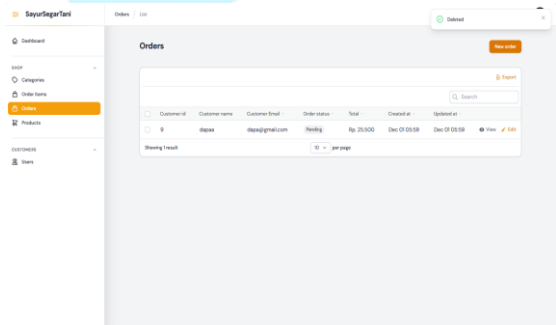
Tabel 3.1 Black Box Testing Halaman Admin

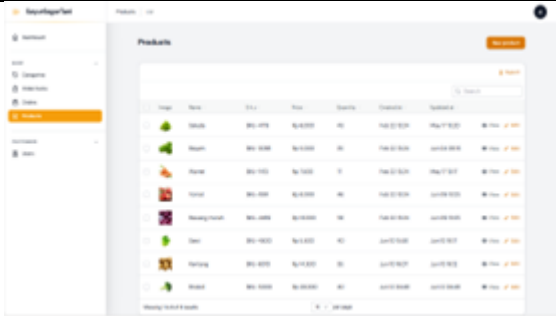
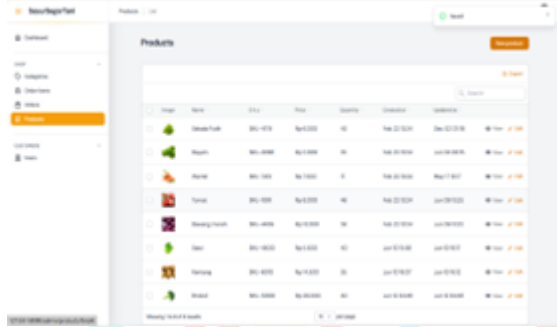
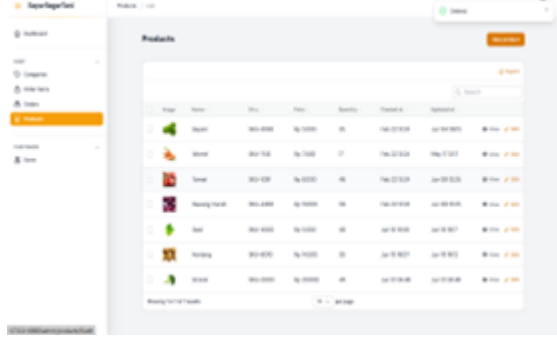
Pengujian 1	
Fitur yang Diuji	Login
Skenario	Admin memasukkan email dan password
Hasil Pengujian	

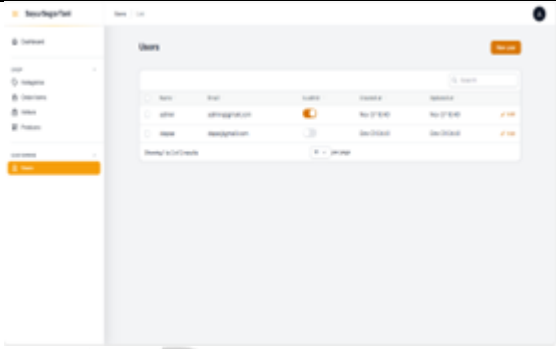
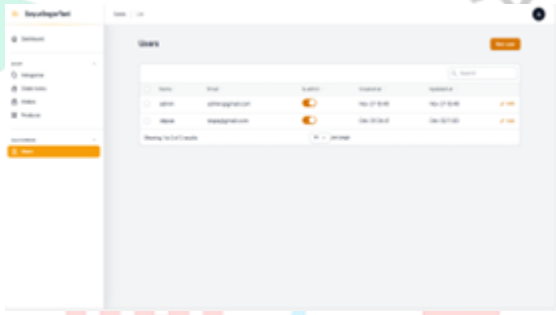
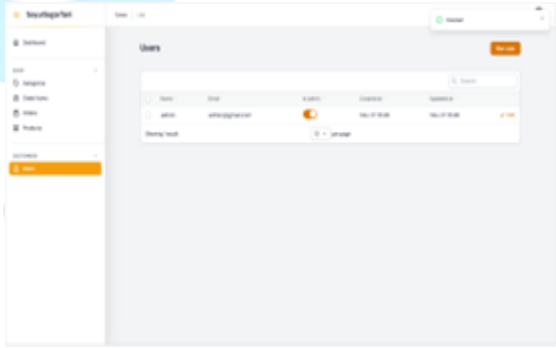
Hasil	Admin berhasil masuk dan diarahkan ke halaman utama
Pengujian 2	
Fitur yang Diuji	Halaman Utama
Skenario	Admin berhasil login
Hasil Pengujian	
Hasil	Dashboard menampilkan produk penjualan dan admin pergi ke dashboard admin
Pengujian 3	
Fitur yang Diuji	Akses Dashboard Admin
Skenario	Admin masuk ke dashboard admin
Hasil Pengujian	
Hasil	Dashboard admin menampilkan Shop untuk penjualan dan user yang sudah login serta jumlah items, penjualan, dan pesanan
Pengujian 4	
Fitur yang Diuji	Categories - New Category
Skenario	Admin mengisi form category lalu klik "save"
Hasil Pengujian	

	
Hasil	Data category tersimpan dan muncul di daftar categories
Pengujian 5	
Fitur yang Diuji	Categories - Edit
Skenario	Admin mengubah data category dengan menekan tombol "edit"
Hasil Pengujian	
Hasil	Data category berhasil diperbarui
Pengujian 6	
Fitur yang Diuji	Categories - Delete
Skenario	Admin memilih category yang ingin dihapus lalu klik "delete" dan muncul konfirmasi hapus lalu klik confirm
Hasil Pengujian	
Hasil	Data category berhasil dihapus dari database
Pengujian 7	
Fitur yang Diuji	Order Items – New Order Items
Skenario	Admin mengisi form new order items lalu klik "simpan"
Hasil Pengujian	

	
Hasil	Data items tersimpan dan muncul di daftar order items
Pengujian 8	
Fitur yang Diuji	Order Items - Edit
Skenario	Admin mengubah data items dengan menekan tombol “edit”
Hasil Pengujian	
Hasil	Data items berhasil diperbarui
Pengujian 9	
Fitur yang Diuji	Order Items - Delete
Skenario	Admin memilih items yang ingin dihapus lalu klik edit dan tekan tombol “delete”
Hasil Pengujian	
Hasil	Data items berhasil dihapus dari database
Pengujian 10	
Fitur yang Diuji	Orders – New Order
Skenario	Admin mengisi form new order lalu klik “simpan”
Hasil Pengujian	

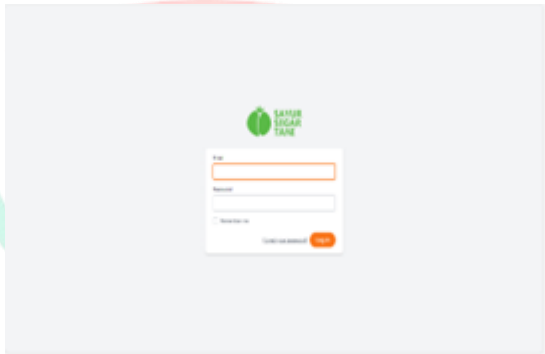
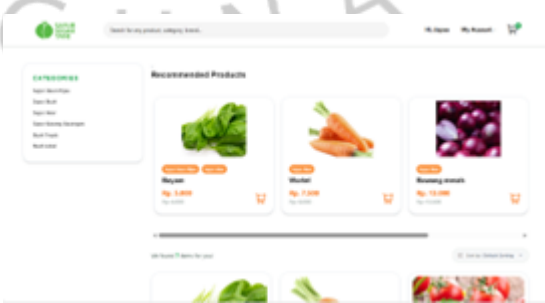
	
Hasil	Data order tersimpan dan muncul di daftar orders
Pengujian 11	
Fitur yang Diuji	Orders - Edit
Skenario	Admin mengubah data orders dengan menekan tombol “edit”
Hasil Pengujian	
Hasil	Data order berhasil diperbarui
Pengujian 12	
Fitur yang Diuji	Orders - Delete
Skenario	Admin memilih items yang ingin dihapus lalu klik edit dan tekan tombol “delete”
Hasil Pengujian	
Hasil	Data order berhasil dihapus dari database
Pengujian 13	
Fitur yang Diuji	Products – New product
Skenario	Admin mengisi form new product lalu klik “simpan”
Hasil Pengujian	

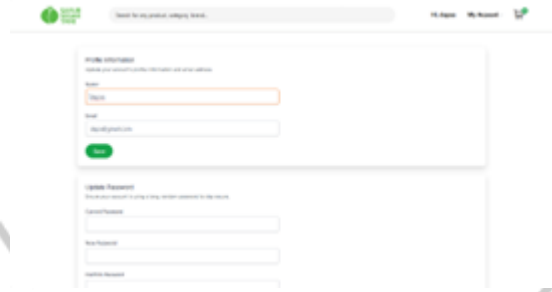
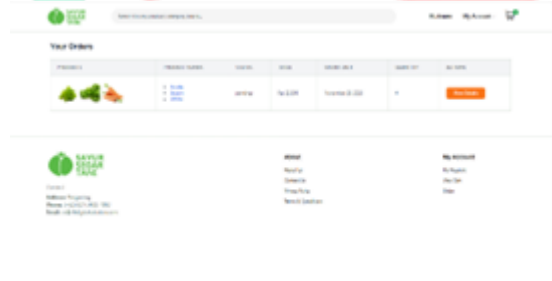
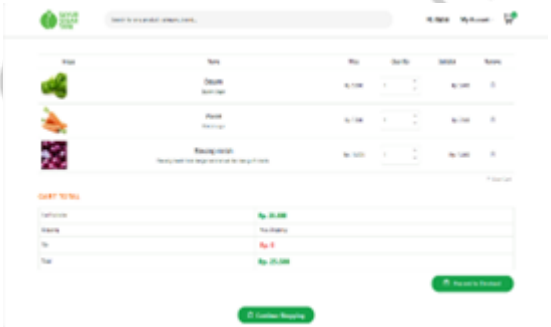
	
Hasil	Data product tersimpan dan muncul di daftar products
Pengujian 14	
Fitur yang Diuji	Products - Edit
Skenario	Admin mengubah data product dengan menekan tombol "edit"
Hasil Pengujian	
Hasil	Data product berhasil diperbarui
Pengujian 15	
Fitur yang Diuji	Product - Delete
Skenario	Admin memilih product yang ingin dihapus lalu klik edit dan tekan tombol "delete"
Hasil Pengujian	
Hasil	Data product berhasil dihapus dari database
Pengujian 16	
Fitur yang Diuji	Users – New user
Skenario	Admin mengisi form new user lalu klik "simpan"
Hasil Pengujian	

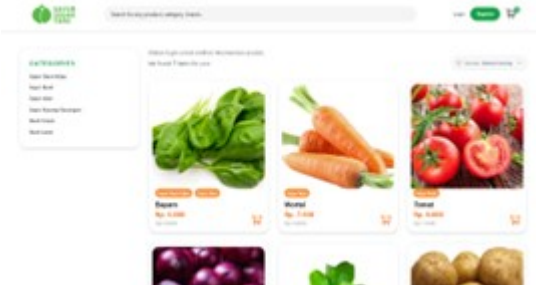
	
Hasil	Data user tersimpan dan muncul di daftar users
Pengujian 17	
Fitur yang Diuji	Users – Edit
Skenario	Admin mengubah data user dengan menekan tombol “edit”
Hasil Pengujian	
Hasil	Data user berhasil diperbarui
Pengujian 18	
Fitur yang Diuji	User - Delete
Skenario	Admin memilih user yang ingin dihapus lalu klik edit dan tekan tombol “delete”
Hasil Pengujian	
Hasil	Data user berhasil dihapus dari database
Pengujian 19	
Fitur yang Diuji	Logout
Skenario	Admin klik menu profile lalu klik sign out

Hasil Pengujian	
Hasil	Admin keluar dari dashboard admin dan diarahkan ke halaman login

Tabel 3.2 Black Box Testing Halaman User

Pengujian 1	
Fitur yang Diuji	Login
Skenario	User memasukkan email dan password
Hasil Pengujian	
Hasil	User berhasil masuk ke Halaman Utama
Pengujian 2	
Fitur yang Diuji	Halaman Utama
Skenario	User berhasil login
Hasil Pengujian	
Hasil	Halaman Utama menampilkan produk yang sedang dijual dengan harga diskon

Pengujian 3	
Fitur yang Diuji	Account Settings
Skenario	User bisa mengubah profile, mengubah password, dan menghapus profile di account settings
Hasil Pengujian	
Hasil	Account setting menampilkan profile pengguna serta bisa mengubah nama, email, dan password pengguna
Pengujian 4	
Fitur yang Diuji	Order Dashboard
Skenario	User bisa melihat produk yang telah diorder
Hasil Pengujian	
Hasil	Order dashboard menampilkan produk yang sudah kita order
Pengujian 5	
Fitur yang Diuji	Cart
Skenario	User bisa melihat produk yang sudah dipesan dan bisa melakukan proses to checkout
Hasil Pengujian	
Hasil	Cart menampilkan produk yang sudah dipesan dan bisa dihapus atau Kembali ke halaman utama untuk berbelanja
Pengujian 6	

Fitur yang Diuji	Payment
Skenario	User membayar produk yang sudah dipesan menggunakan transaksi online
Hasil Pengujian	
Hasil	Payment menampilkan transaksi pembayaran via online dan detail produk yang sudah dipesan
Pengujian 7	
Fitur yang Diuji	Logout
Skenario	User klik menu logout di My Account
Hasil Pengujian	
Hasil	User keluar dan diarahkan ke menu utama tanpa login

3.2.6 Perancangan Database

Database PureGreens dirancang untuk menyimpan data utama yang terkait dengan pengguna, produk, transaksi, dan detail pesanan. Setiap tabel memiliki fungsi berbeda namun saling terhubung melalui foreign key untuk memastikan integritas data. Berikut adalah perancangan database pada sistem PureGreens.

Tabel 3.3 Tabel User

No	Field	Tipe Data	Keterangan
1	id	bigint	Primary key
2	name	varchar(255)	Nama pengguna
3	email	varchar(255)	Email pengguna (unik)

4	email_verified_at	timestamp	Waktu verifikasi email
5	password	varchar(255)	Password yang sudah di-hash
6	remember_token	varchar(100)	Token autentikasi
7	created_at	timestamp	Waktu dibuat
8	updated_at	timestamp	Waktu diubah

Tabel users digunakan untuk menyimpan informasi akun pengguna yang mengakses website PureGreens. Tabel ini berperan penting dalam proses login, registrasi, dan pengelolaan profil.

Tabel 3.4 Tabel Products

No	Field	Tipe Data	Keterangan
1	id	bigint	Primary key
2	name	varchar(255)	Nama produk
3	description	text	Deskripsi produk
4	price	decimal(10,2)	Harga produk
5	stock	int	Stok produk
6	image	varchar(255)	Lokasi file gambar produk
7	created_at	timestamp	Waktu dibuat
8	updated_at	timestamp	Waktu diubah

Tabel products menyimpan daftar produk buah dan sayuran yang ditampilkan pada halaman Home, Rekomendasi, dan Keranjang. Setiap produk memiliki informasi harga, stok, deskripsi, dan gambar.

Tabel 3.5 Tabel Categories

No	Field	Tipe Data	Keterangan
1	id	bigint	Primary key
2	name	varchar(255)	Nama kategori produk
3	created_at	timestamp	Waktu dibuat
4	updated_at	timestamp	Waktu diubah

Tabel categories menyimpan daftar kategori seperti “Sayur Daun”, “Sayur Akar”, dan “Buah Lokal”. Kategori digunakan pada sidebar halaman Home dan membantu pengguna melakukan filtering.

Tabel 3.6 Tabel Categories Product

No	Field	Tipe Data	Keterangan
1	category_id	bigint	Foreign key ke tabel categories
2	product_id	bigint	Foreign key ke tabel products

Tabel category_product menghubungkan produk dengan kategori. Karena satu produk dapat memiliki lebih dari satu kategori, tabel ini berfungsi sebagai jembatan many-to-many.

Tabel 3.7 Tabel Orders

No	Field	Tipe Data	Keterangan
1	id	bigint	Primary key
2	user_id	bigint	Foreign key ke tabel users
3	total_amount	decimal(10,2)	Total pembayaran
4	status	varchar(50)	Status pesanan
5	created_at	timestamp	Waktu dibuat
6	updated_at	timestamp	Waktu diubah

Tabel orders menyimpan data pesanan pengguna, termasuk total harga dan status transaksi. Data pada tabel ini ditampilkan pada halaman Riwayat Pesanan.

Tabel 3.8 Tabel Orders Item

No	Field	Tipe Data	Keterangan
1	id	bigint	Primary key
2	order_id	bigint	Foreign key ke tabel orders
3	product_id	bigint	Foreign key ke tabel products
4	quantity	int	Jumlah yang dipesan
5	price	decimal(10,2)	Harga per item
6	created_at	timestamp	Waktu dibuat
7	updated_at	timestamp	Waktu diubah

Tabel order_items mencatat rincian produk dalam setiap pesanan. Data ini digunakan untuk menampilkan detail riwayat dan menghitung total pembelian.

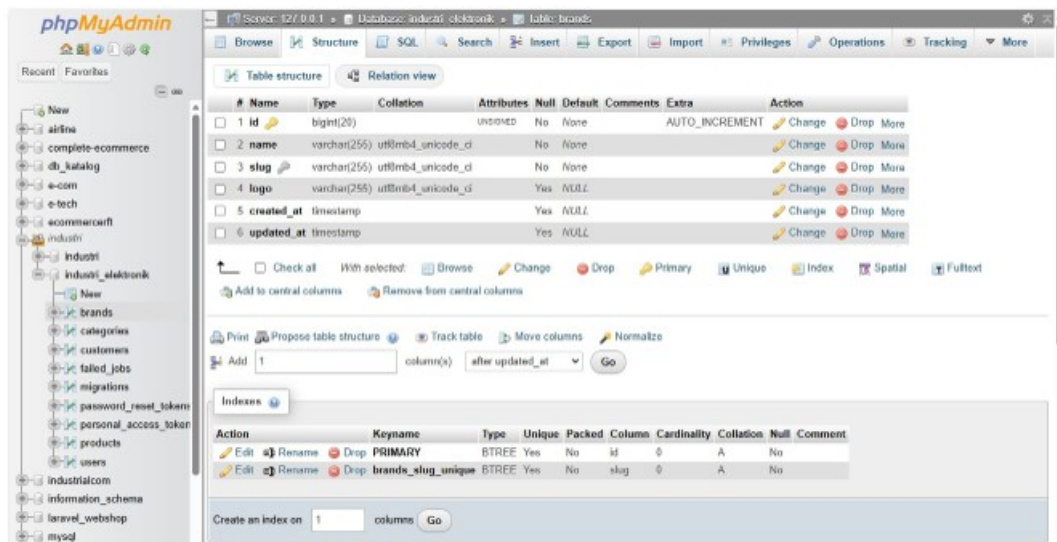
Tabel 3.9 Tabel Billing Details

No	Field	Type Data	Keterangan
1	id	bigint	Primary key
2	order_id	bigint	Foreign key ke tabel orders
3	full_name	varchar(255)	Nama penerima
4	address	varchar(255)	Alamat lengkap
5	phone_number	varchar(20)	Nomor telepon
6	city	varchar(255)	Kota
7	created_at	timestamp	Waktu dibuat

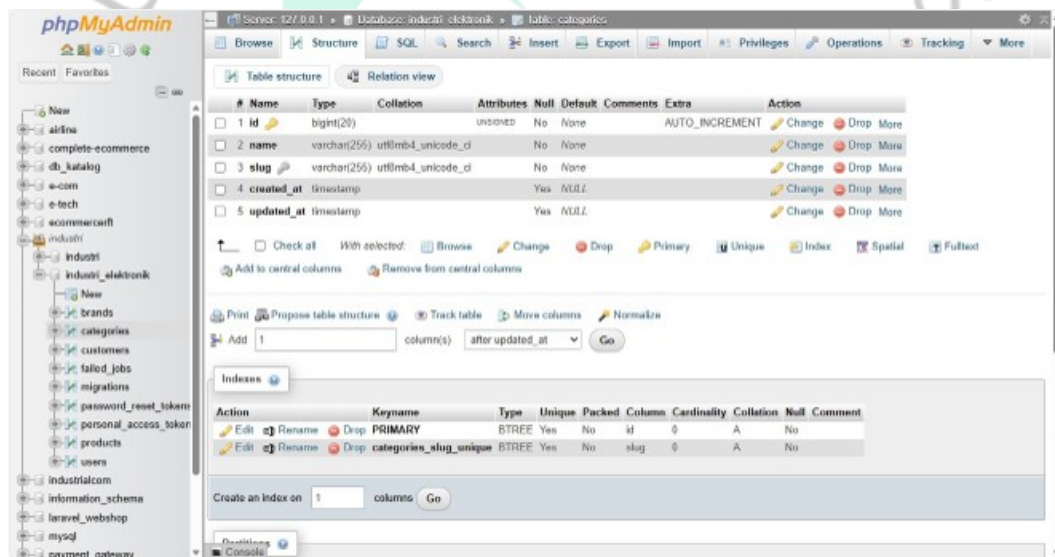
Tabel `billing_details` menyimpan informasi alamat dan kontak penerima yang diperlukan saat pengguna melakukan checkout. Tabel ini berfungsi melengkapi proses transaksi.

3.2.6 Realisasi Database

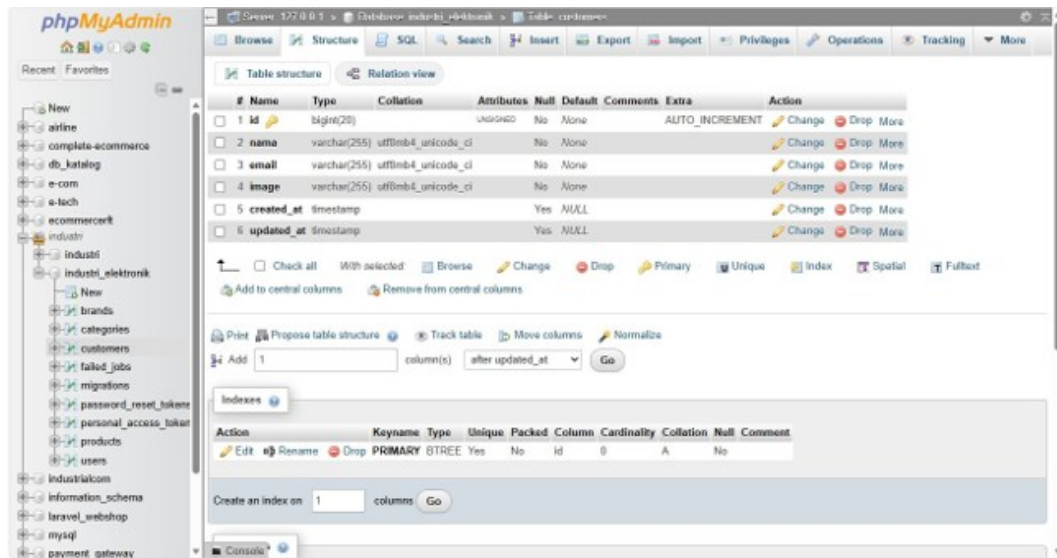
Pada tahap ini, setelah praktikan membuat sketsa basis data, langkah selanjutnya adalah melakukan realisasi basis data. Basis data pada website *E-Commerce* sayur-sayuran dikembangkan menggunakan tools XAMPP yang terintegrasi dengan *MySQL* sebagai sistem manajemen basis data. Pemilihan tools ini disesuaikan dengan mempertimbangkan kemudahan konfigurasi, stabilitas, dan kompatibilitasnya dengan *framework Laravel* yang digunakan dalam pengembangan *backend*. Selain itu, XAMPP dan *MySQL* juga banyak digunakan dalam pengembangan aplikasi berbasis web pada proyek lainnya.



Gambar 3.28 Table brands



Gambar 3.29 Table kategori



Gambar 3.30 Table customer

Gambar di atas menunjukkan realisasi tabel basis data yang telah dibuat oleh praktikan menggunakan tools XAMPP dengan sistem manajemen basis data MySQL. Pada basis data tersebut terdapat beberapa tabel utama yang digunakan dalam website E-Commerce sayur-sayuran yang berfungsi untuk menyimpan data pengguna dan produk penjualan. Selain itu, terdapat juga beberapa tabel tambahan yang merupakan bawaan dari framework Laravel, seperti migrations, password_resets, personal_access dan failed_jobs yang digunakan untuk mendukung pengelolaan sistem dan keamanan aplikasi

3.3 Kendala Yang Dihadapi

Berdasarkan pelaksanaan Kerja Profesi di PT Reliable Future Technology, terdapat beberapa kendala ringan yang ditemui selama proses pengembangan frontend website PureGreens. Adapun kendala yang dialami antara lain:

- Adaptasi terhadap alur kerja tim, terutama dalam memahami standar coding yang digunakan perusahaan.
- Koordinasi dengan bagian backend terkadang membutuhkan waktu tambahan untuk menunggu ketersediaan data atau endpoint tertentu.
- Pembagian waktu kerja antara pengerjaan tugas KP dan kegiatan akademik lainnya sehingga perlu manajemen waktu yang lebih baik.

3.4 Cara Mengatasi Kendala

Untuk mengatasi kendala-kendala tersebut, praktikan melakukan beberapa langkah penyelesaian, baik secara mandiri maupun melalui bantuan rekan tim, yaitu sebagai berikut:

- a. Meningkatkan koordinasi dengan tim melalui komunikasi rutin dan mengikuti alur kerja yang diterapkan perusahaan, sehingga adaptasi terhadap standar yang digunakan menjadi lebih cepat.
- b. Melakukan sinkronisasi berkala dengan backend untuk memastikan data yang dibutuhkan sudah tersedia, serta menyesuaikan pekerjaan frontend agar tetap dapat berjalan tanpa hambatan.
- c. Mengatur waktu kerja secara lebih terencana, seperti menetapkan target harian dan membagi waktu antara KP dan kegiatan kampus, sesuai dengan prinsip time management dalam lingkungan kerja profesional.